Advanced Machine learning Mastering Course

Introduced by

George Samuel

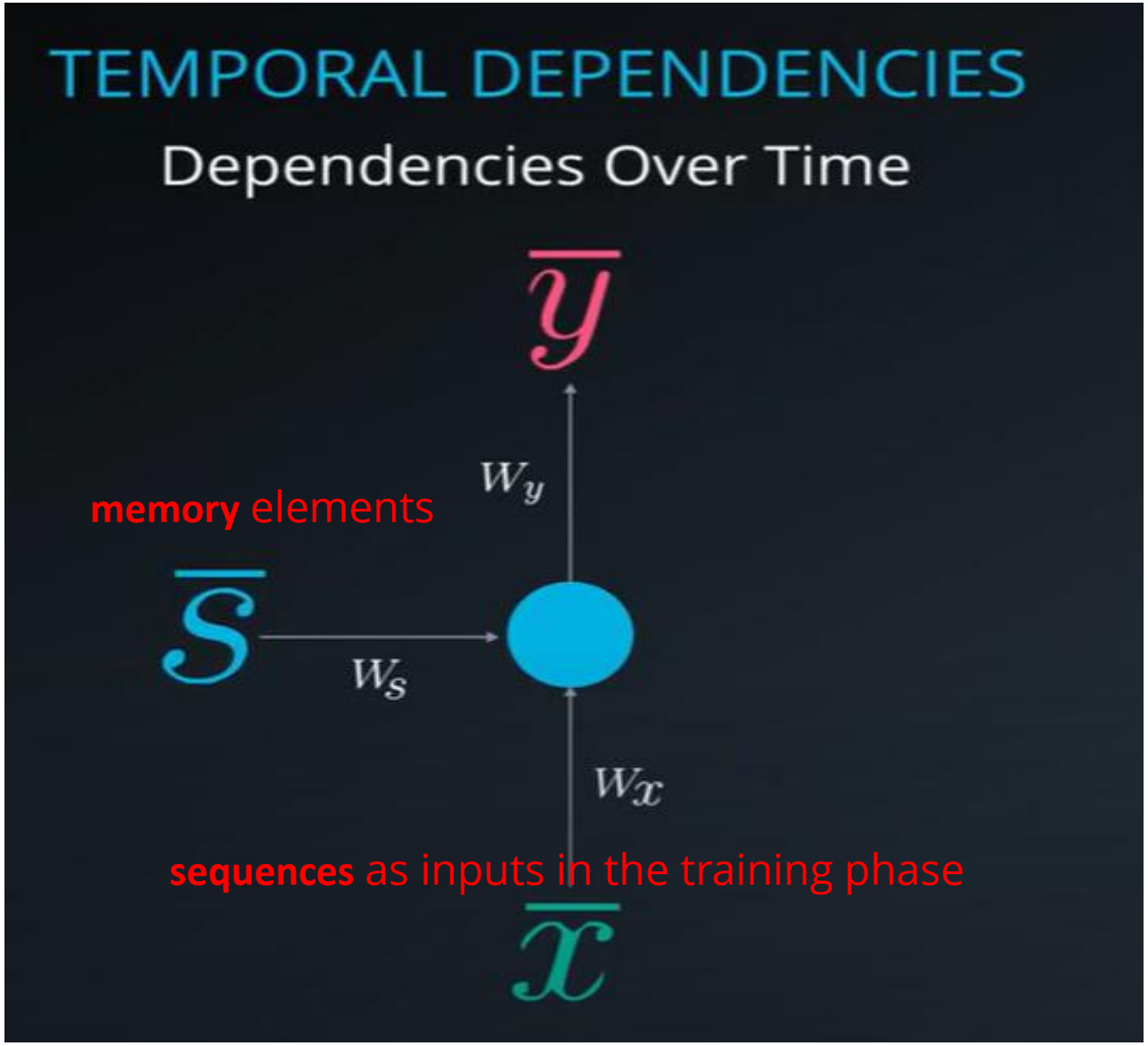
Recurrent Neural Network

GRU-LSTM

Innovisionray.com

2024

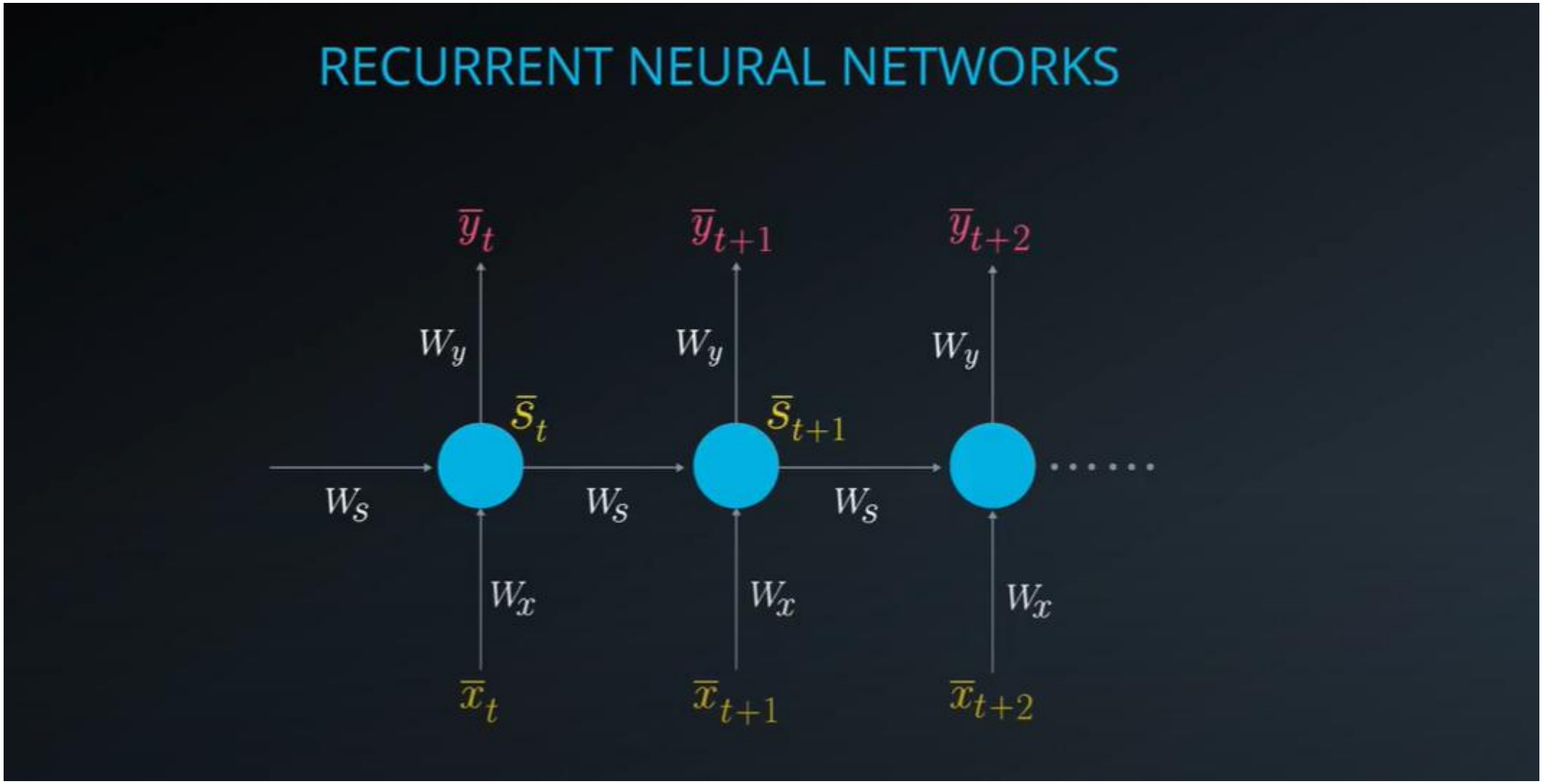
Recurrent Neural Network



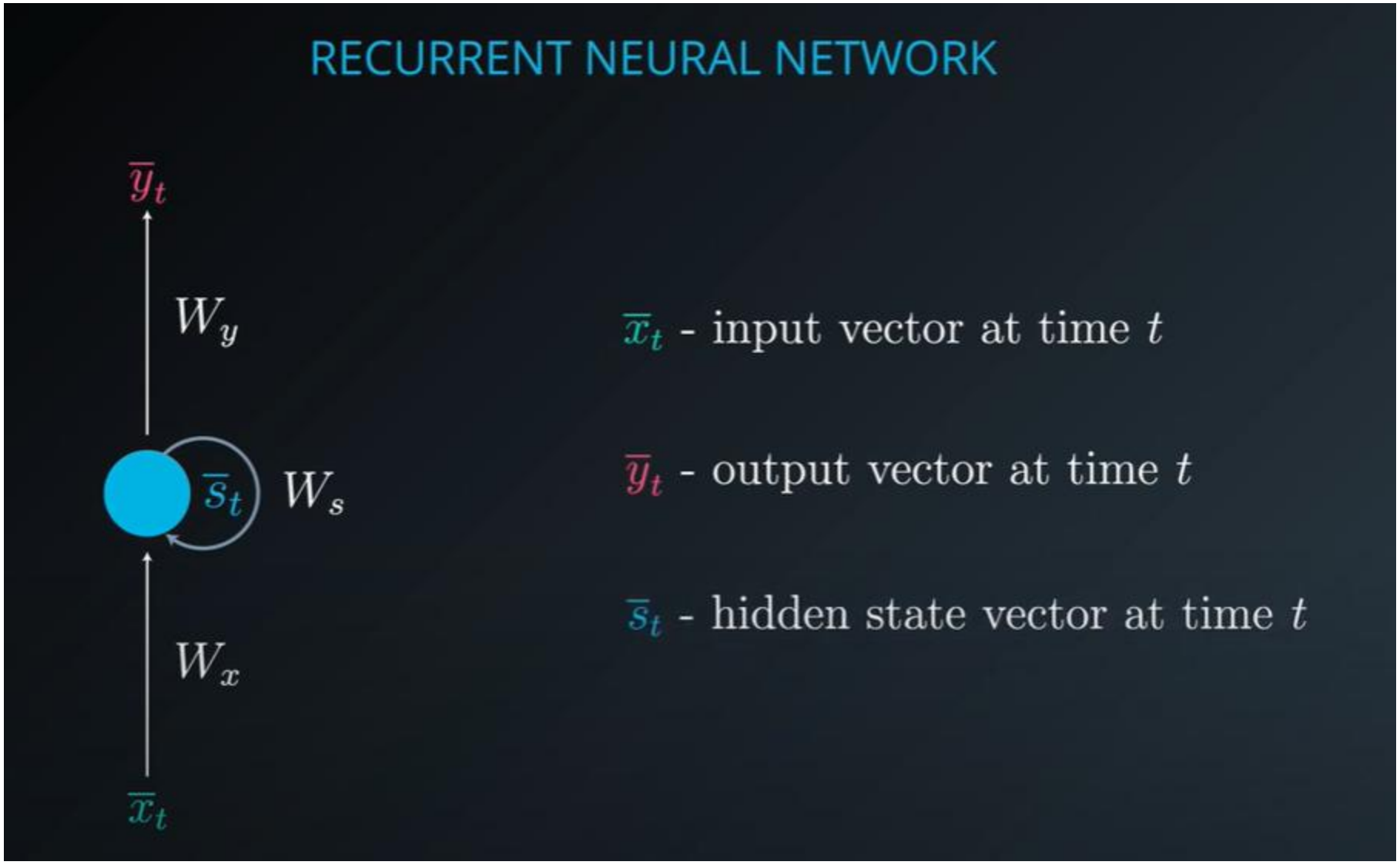
Recurrent Neural Network

- Sentiment Analysis
- Speech Recognition
- Time Series Prediction
- Natural Language Processing
- Gesture Recognition

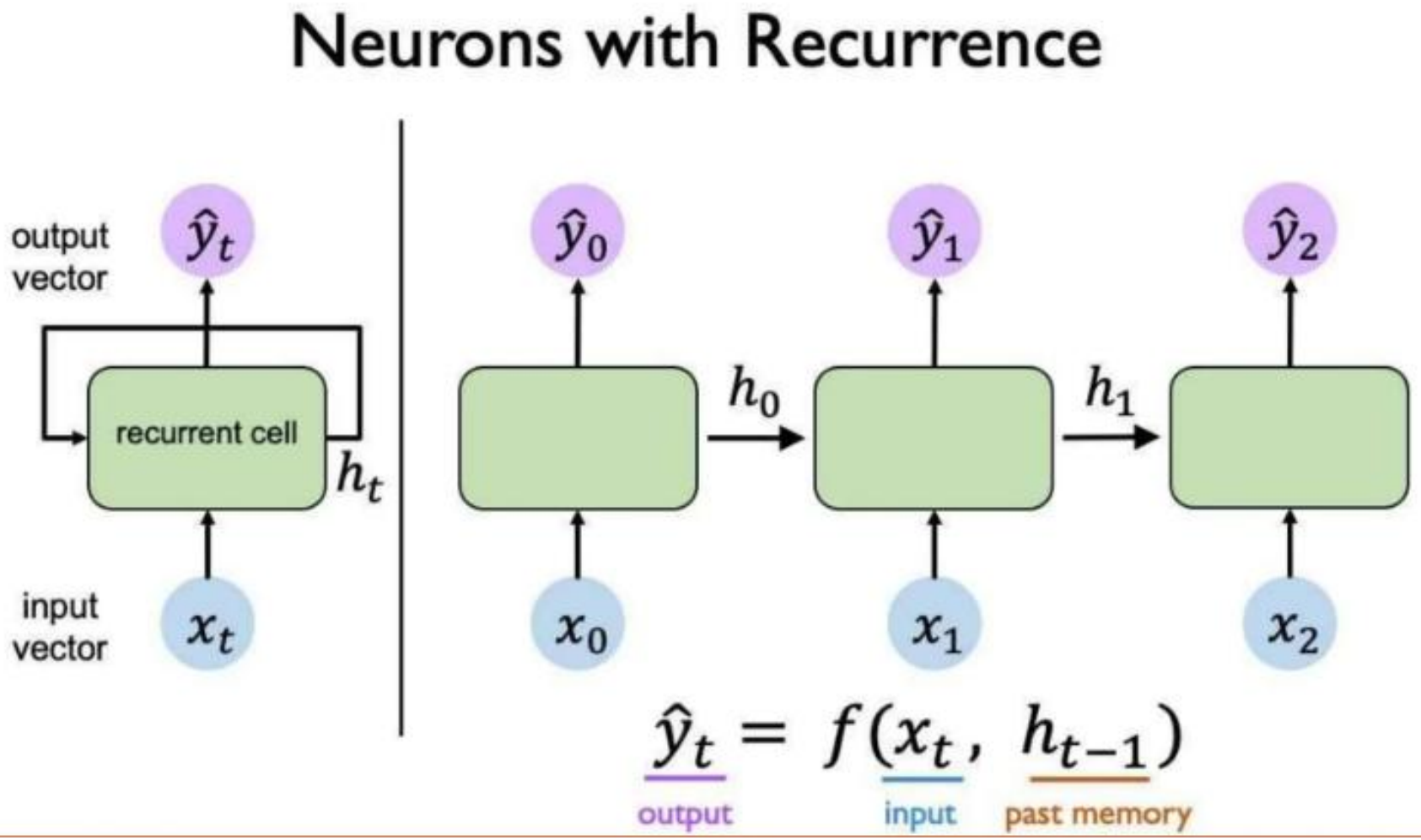
Recurrent Neural Network



Recurrent Neural Network



Recurrent Neural Network



Recurrent Neural Network

RECURRENT NEURAL NETWORK (RNN)

Recurrent: perform same task for every element of a sequence
Output: depend on:
previous computations as well as
new inputs
*RNNs have a "**memory**" of past !*

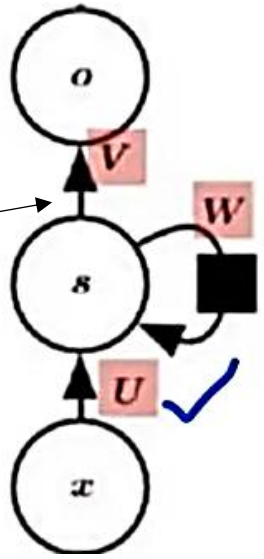
current state = f(previous state, input)

$$s_t = f(s_{t-1}, x_t) = \tanh(Ws_{t-1} + Ux_t)$$

$$o_t = Vs_t$$

Output of the previous state

need

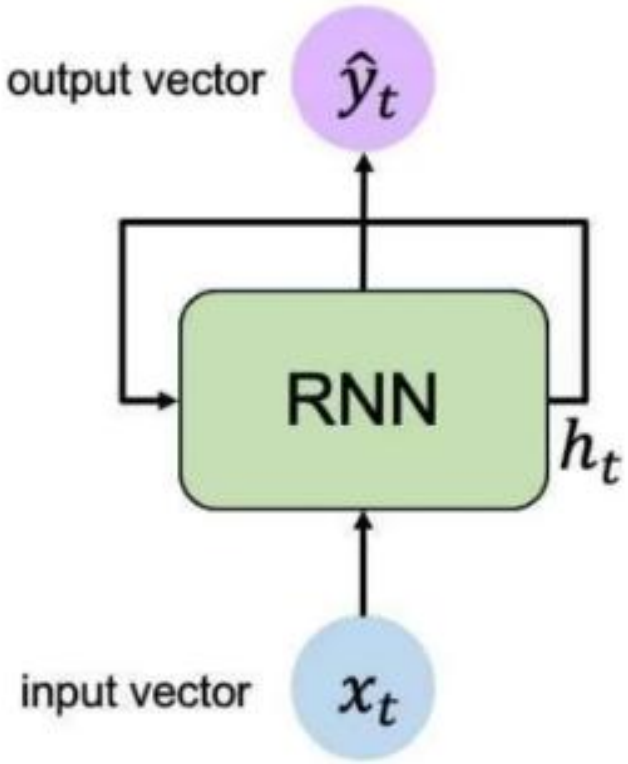


to learn AI

I need to learn AI

Recurrent Neural Network

Recurrent Neural Networks (RNNs)



Apply a **recurrence relation** at every time step to process a sequence:

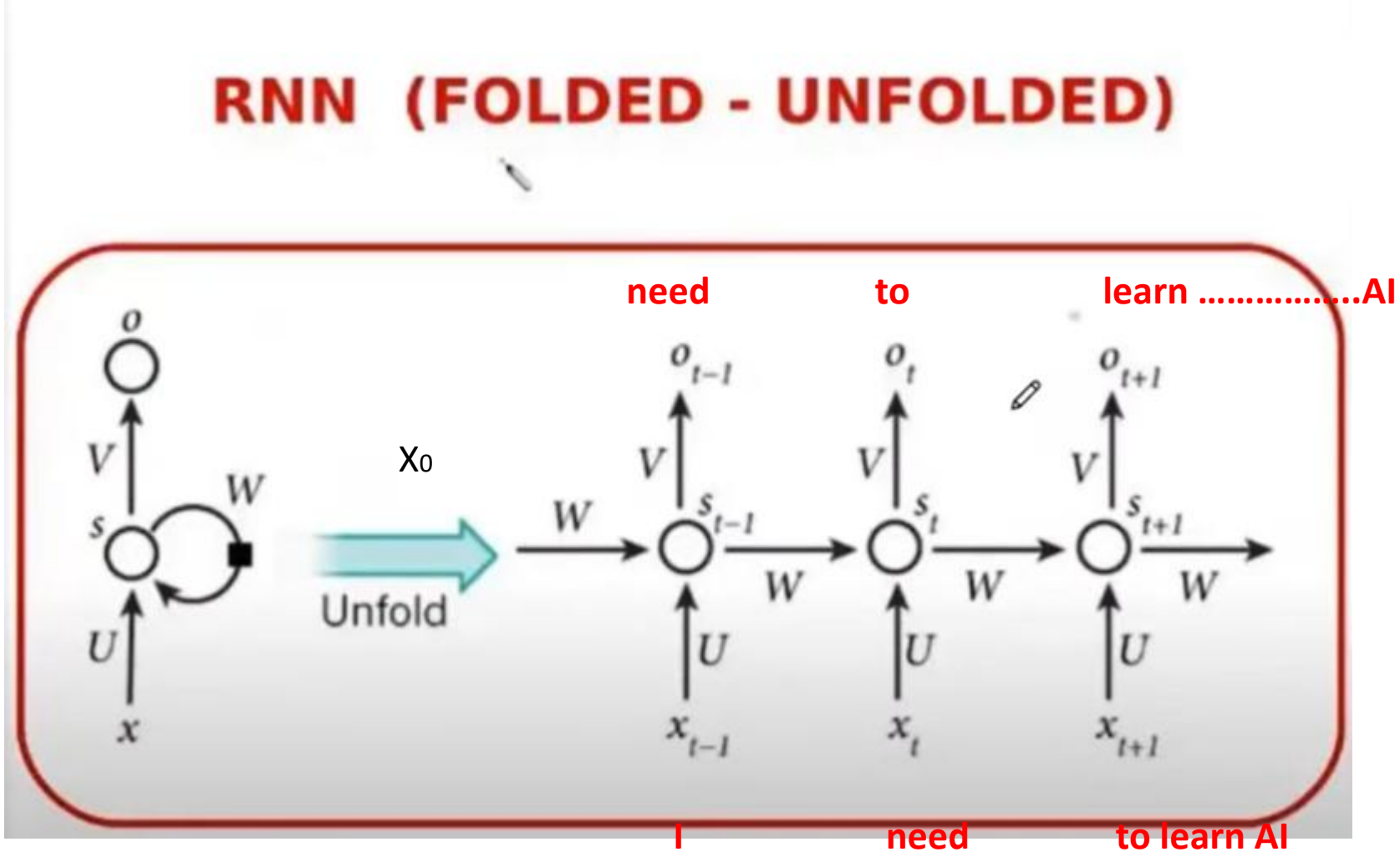
$$\boxed{h_t} = \boxed{f_W}(\boxed{x_t}, \boxed{h_{t-1}})$$

cell state — function with weights W — input — old state

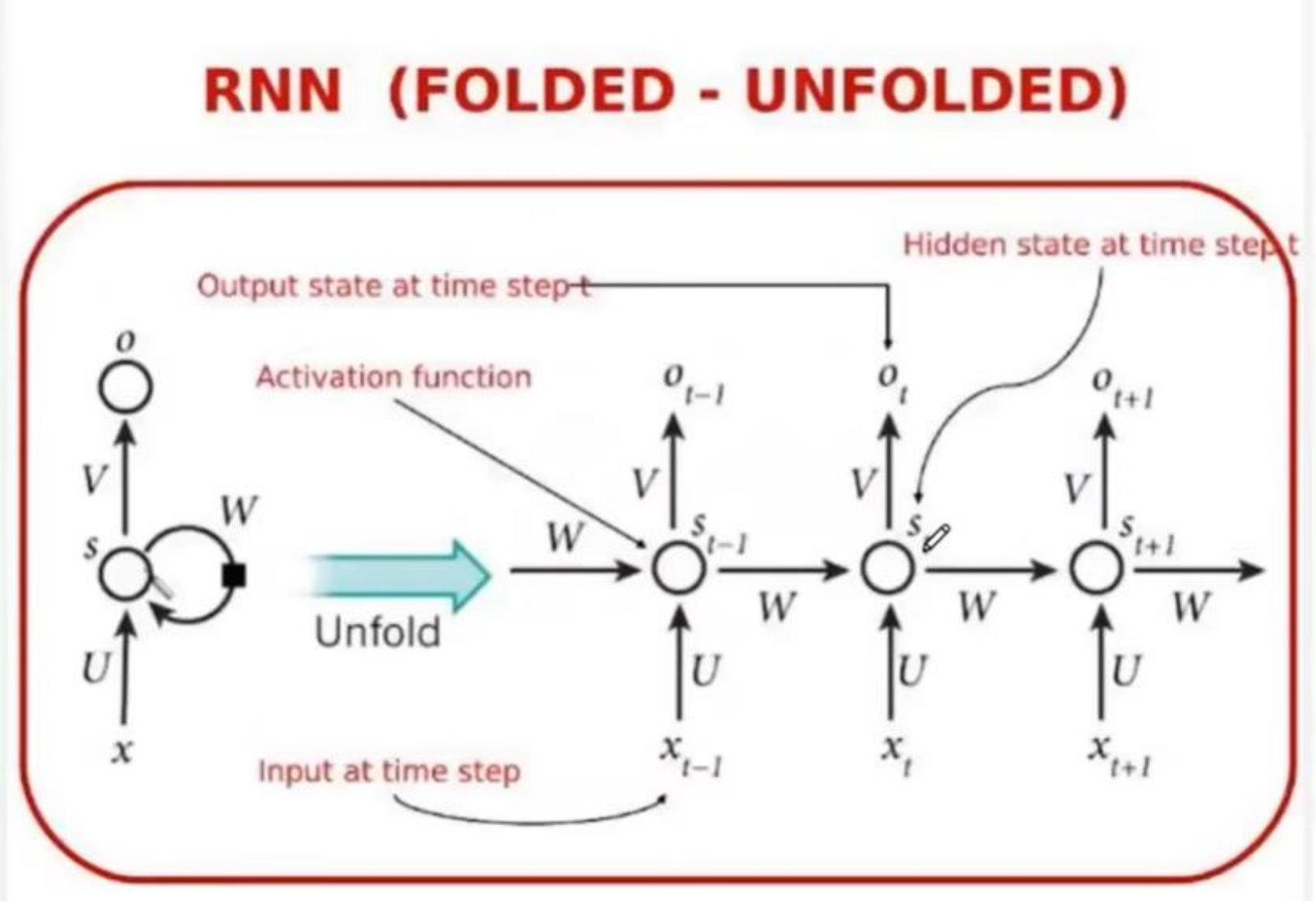
Note: the same function and set of parameters are used at every time step

RNNs have a **state**, h_t, that is updated **at each time step** as a sequence is processed

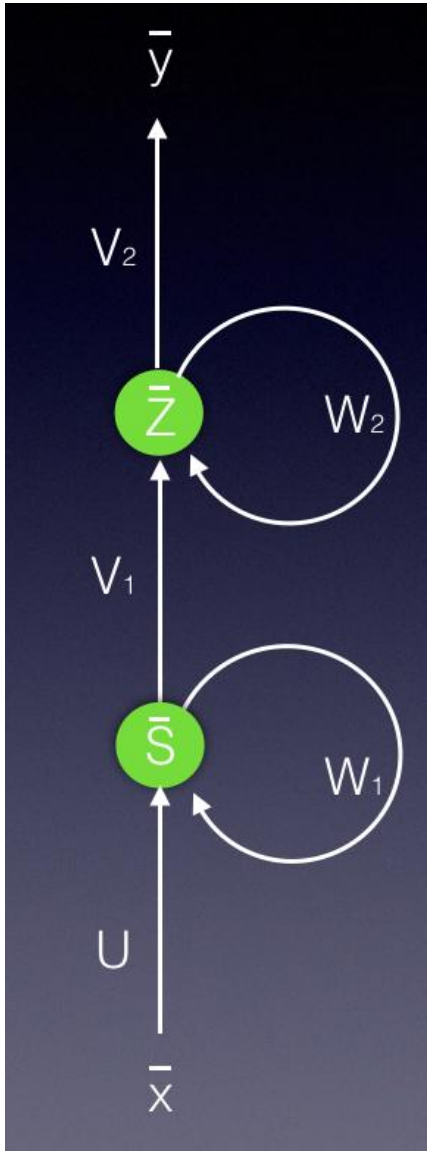
Recurrent Neural Network

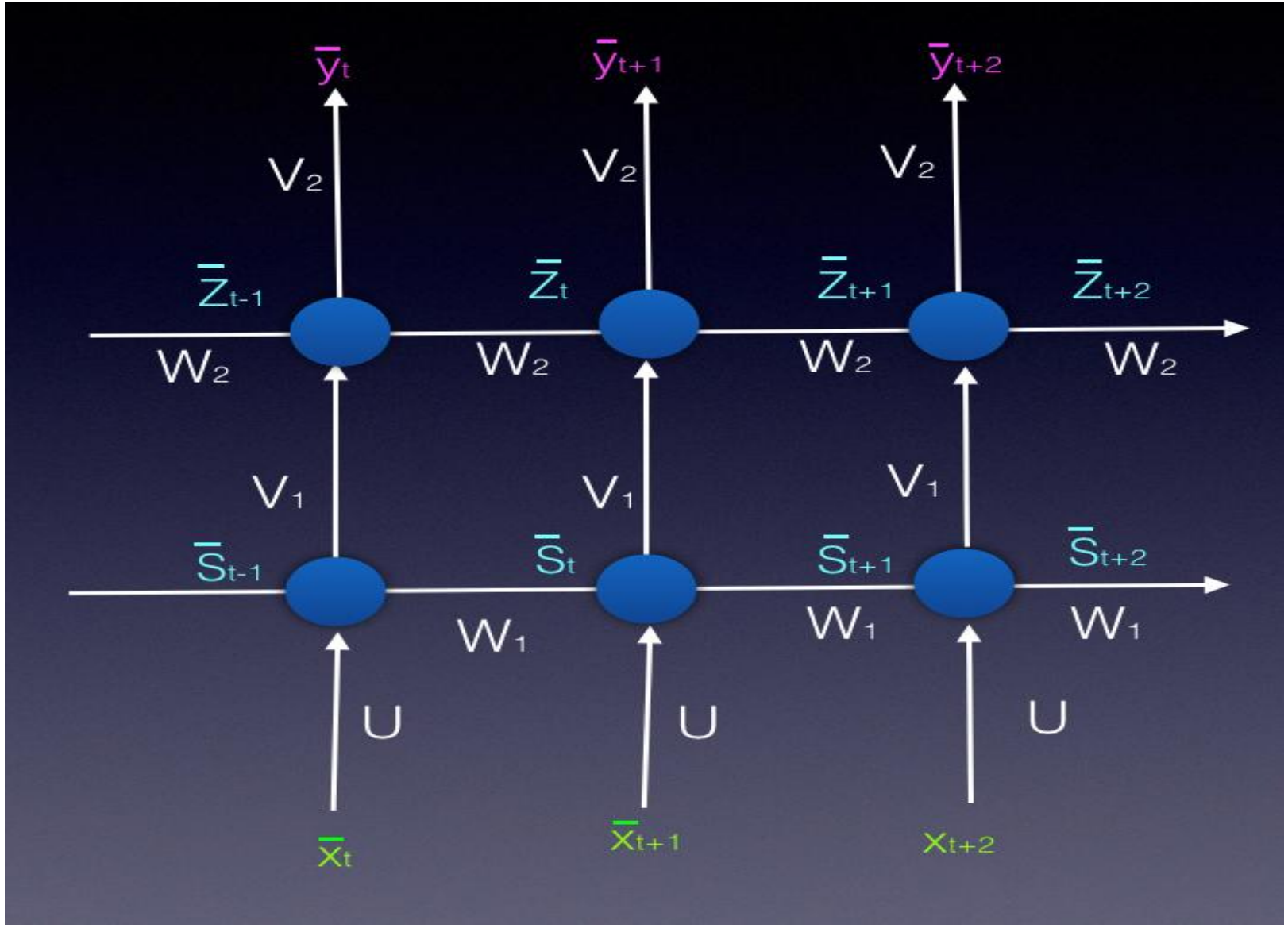


Recurrent Neural Network



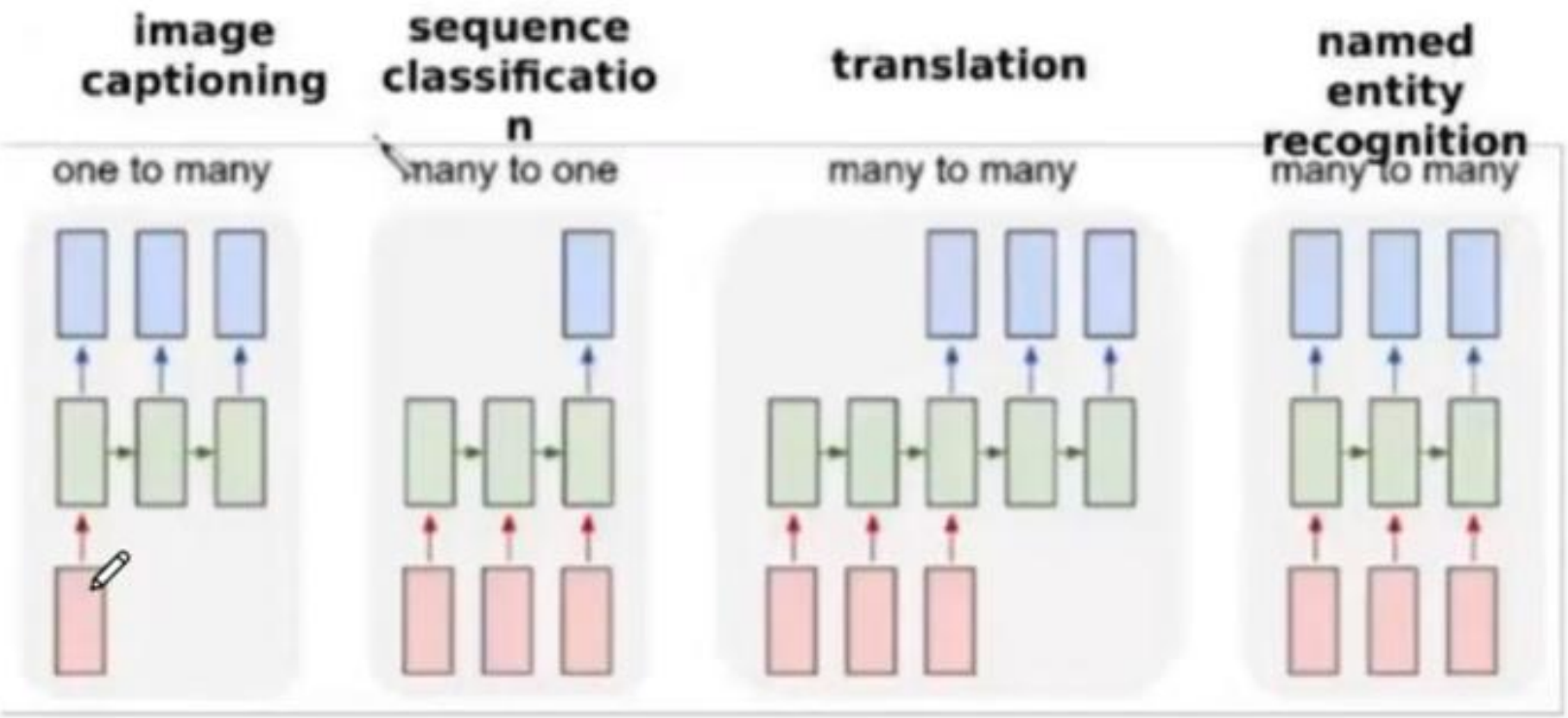
Recurrent Neural Network





Recurrent Neural Network

Examples of Sequence Application of RNN - LSTM



CHARACTER-LEVEL LANGUAGE MODEL

- Vocabulary:
  - Only 4 characters [h, e, l, o]
- Training sequence **"hello"**

1. Apply input with 4 ×1 vocabulary vector sequentially
2. Calculate hidden states: $s_t = \tanh(Ws_{t-1} + Ux_t)$
3. Calculate output: $o_t = Vs_t$
4. Compare with next character & calculate loss
5. Back-propagate errors and updates weights(U,V,W)

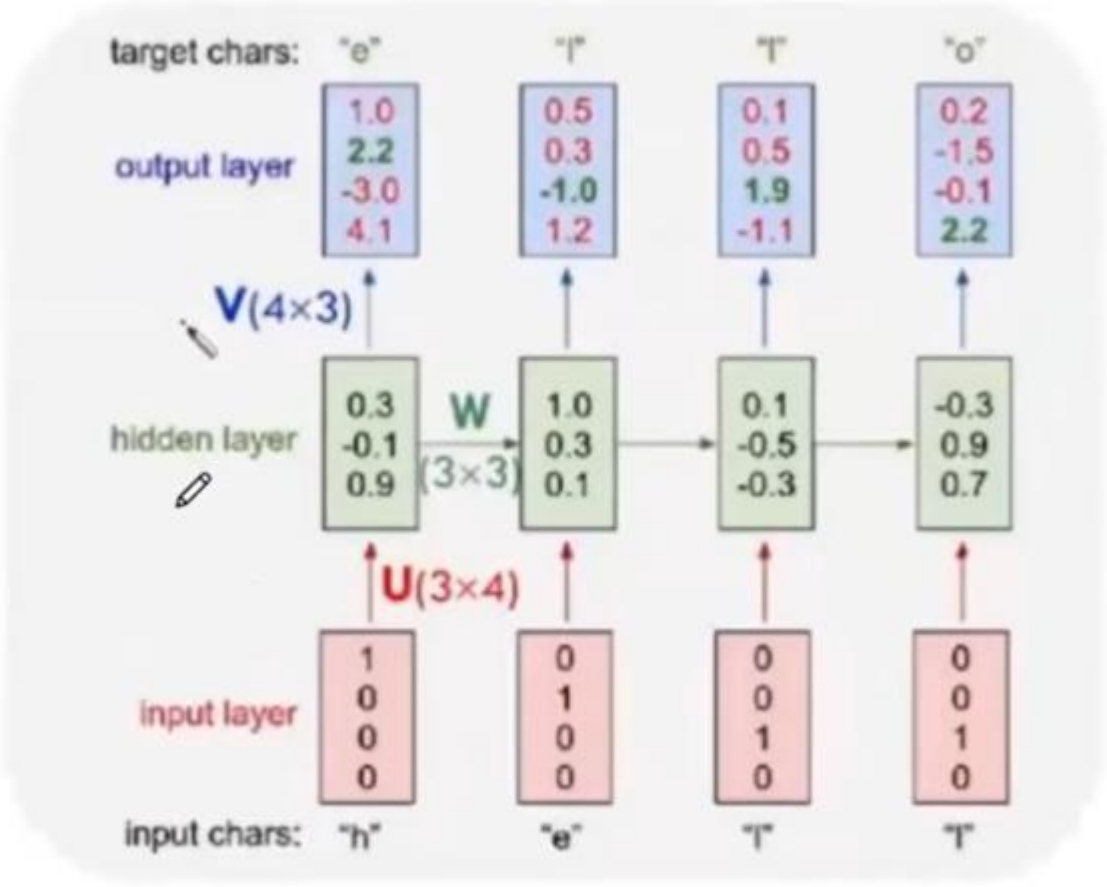
One-HOT Vectors input for Word Sequence

- A vector with all zeros except for the active dimension
- 12 words in a sequence → 12 One-hot vectors
- After the one-hot vectors apply an embedding (Word2Vec, GloVE)

Vocabulary	One-hot vectors			
I	1	0	0	0
am	0	1	0	0
Bond	0	0	1	0
James	0	0	0	1
tired	0	0	0	0
.	0	0	0	0
McGuire	0	0	0	0
!	0	0	0	0

Recurrent Neural Network

CHARACTER-LEVEL LANGUAGE MODEL

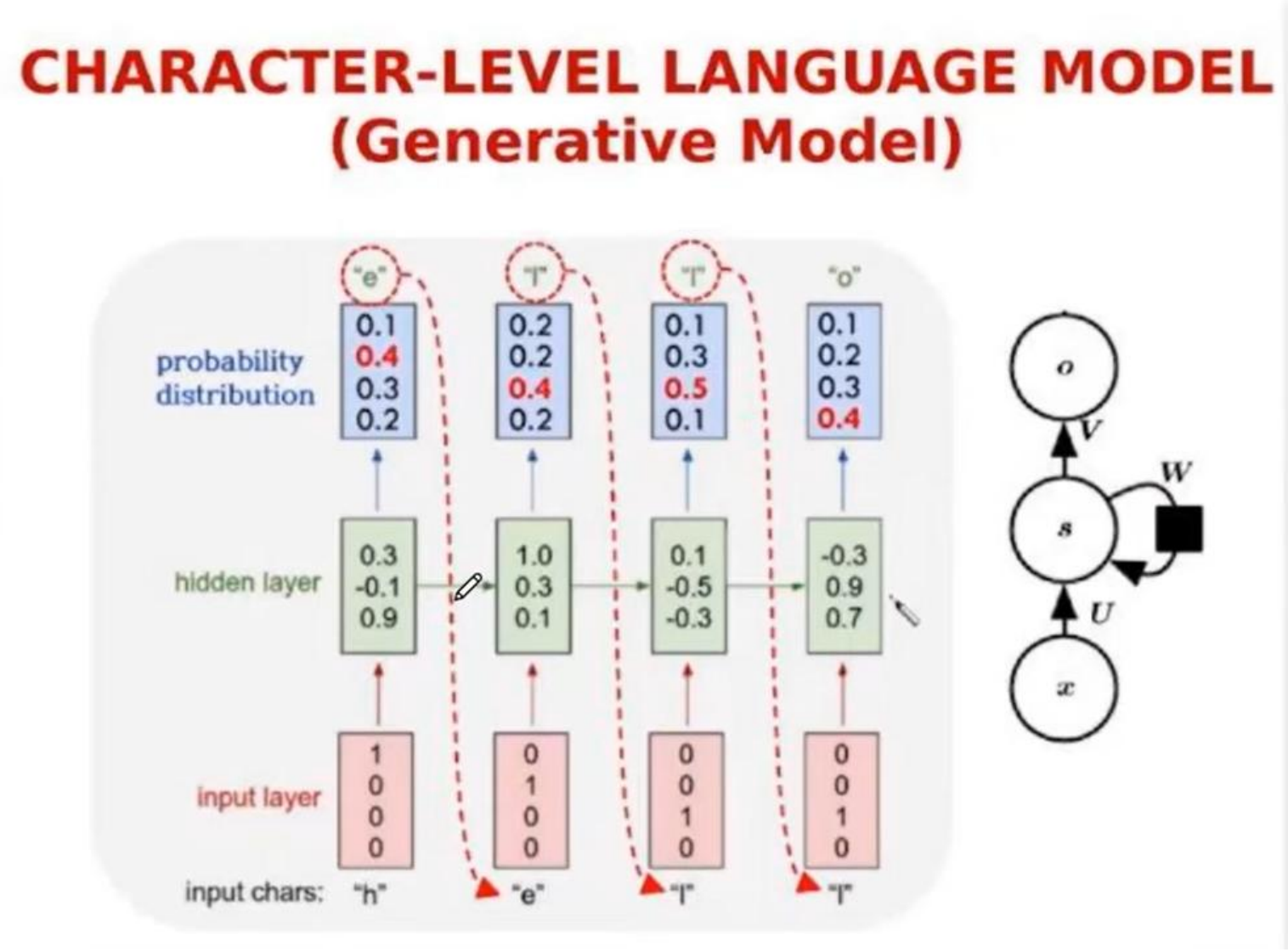


CHARACTER-LEVEL LANGUAGE MODEL

- Training
  - Softmax classifier on output vector
  - Use a cross-entropy loss function
  - Back Propagation Through Time(**BPTT**)
- Test

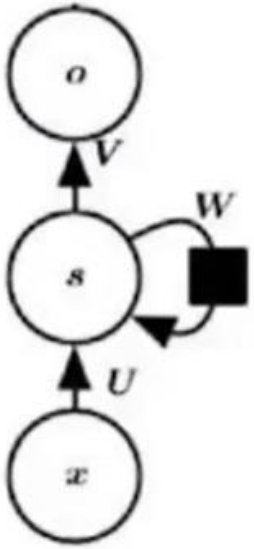
1. Feed a character into the RNN
2. Get a distribution over output
3. Find character with highest probability
4. Feed the highest into RNN input

Recurrent Neural Network



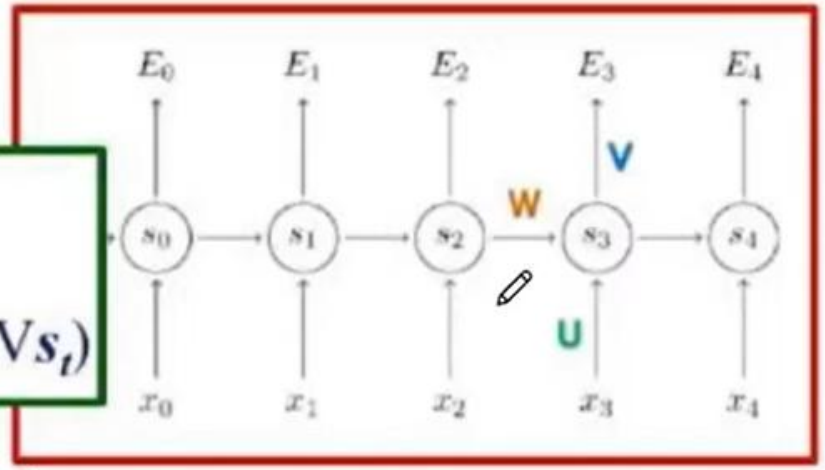
Simple and Real RNN (Number of Parameters)

	Simple RNN	Real RNN
Vocabulary	4	80
Hidden Layers	1	3
Hidden nodes	3	100
U	3×4	100×80
W	3×3	100×100×3
W_{hh}	-	100×100×2
V	4×3	80×100
Total parameters	**33**	**66,000**



Recurrent Neural Network

BACKPROPAGATION THROUGH TIME (BPTT)



- Forward calculation
  - $s_t = \tanh(Ws_{t-1} + Ux_t)$
  - $\hat{y}_t = \text{softmax}(o_t) = \text{softmax}(Vs_t)$

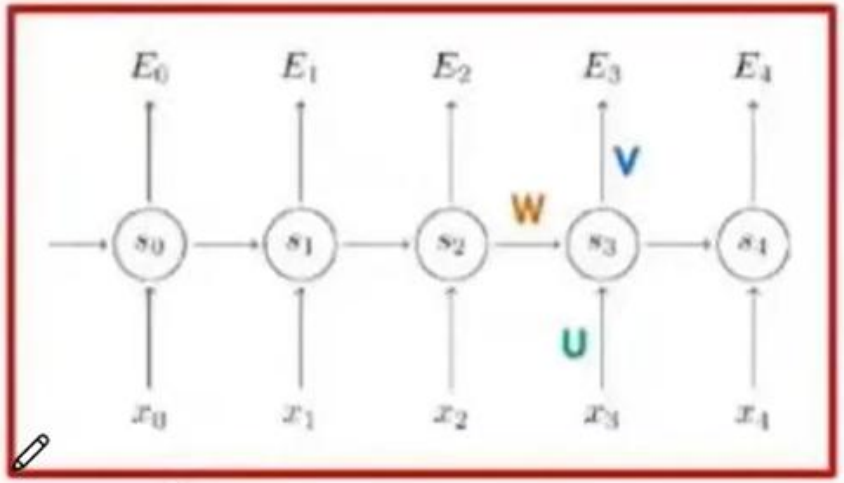
y_t: correct output
$\hat{y}_t$: calculated output of the network

- With cross entropy loss
  - $E_t(y_t, \hat{y}_t) = -y_t \log \hat{y}_t$
  - $E(y, \hat{y}) = \sum_t E_t(y_t, \hat{y}_t) = -\sum_t y_t \log \hat{y}_t$
- Training with a full sentence as one example

BACKPROPAGATION THROUGH TIME (BPTT)

Calculate gradients of error with respect to: **U, V, W.**

$$s_3 = \tanh(Ws_2 + Ux_3)$$
$$s_2 = \tanh(Ws_1 + Ux_2)$$
$$s_1 = \tanh(Ws_0 + Ux_1)$$



Sum up the gradients at each time step

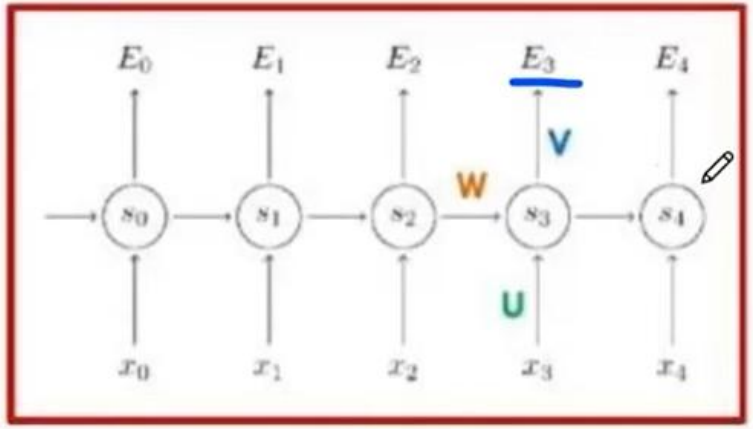
$$\frac{\partial E}{\partial U} = \sum_t \frac{\partial E_t}{\partial U}, \quad \frac{\partial E}{\partial V} = \sum_t \frac{\partial E_t}{\partial V}, \quad \frac{\partial E}{\partial W} = \sum_t \frac{\partial E_t}{\partial W}$$

Recurrent Neural Network

BACKPROPAGATION THROUGH TIME (BPTT)

Calculate Gradients by chain Rule

$$\frac{\partial E_3}{\partial V} = \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial o_3} \cdot \frac{\partial o_3}{\partial V}$$



Remember:

$$E_t(y_t, \hat{y}_t) = -\, y_t \log \hat{y}_t$$

$$\hat{y}_t = \text{softmax}(o_t) = \text{softmax}(Vs_t)$$

$$o_t = Vs_t$$

$$s_t = \tanh(Ws_{t-1} + Ux_t)$$

$$s_3 = \tanh(Ws_2 + Ux_3)$$
$$s_2 = \tanh(Ws_1 + Ux_2)$$
$$s_1 = \tanh(Ws_0 + Ux_1)$$

Softmax fn

$$\hat{y}_k = \frac{e^{a_k}}{\sum_{k'=1}^{K} e^{a_{k'}}} \quad \text{for } k = 1 \text{ to } k = K.$$

Recurrent Neural Network

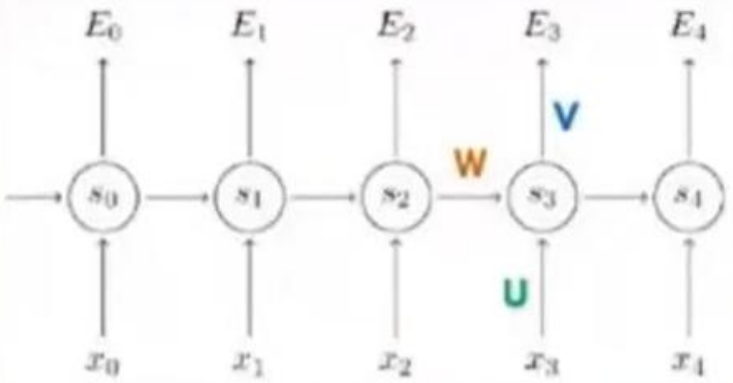
BACKPROPAGATION THROUGH TIME (BPTT)

$$\frac{\partial E_3}{\partial W} = \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial W}$$

$$\frac{\partial s_3}{\partial W} \; ?$$

S_1 and S_2 Depends on W and U too

$$\frac{\partial s_3}{\partial W} \rightarrow \sum_k \frac{\partial s_3}{\partial s_k} \cdot \frac{\partial s_k}{\partial W}$$



$$\frac{\partial E_3}{\partial W} = \sum_k \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_k} \cdot \frac{\partial s_k}{\partial W}$$

$$s_3 = \tanh(Ws_2 + Ux_3)$$
$$s_2 = \tanh(Ws_1 + Ux_2)$$
$$s_1 = \tanh(Ws_0 + Ux_1)$$

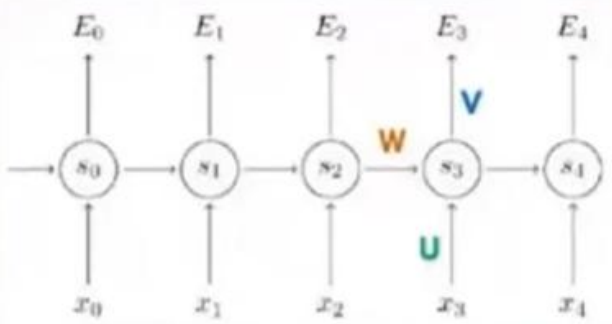
$$\frac{\partial E_3}{\partial W} = \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial W} + \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_2} \cdot \frac{\partial s_2}{\partial W} + \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_1} \cdot \frac{\partial s_1}{\partial W} + \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_0} \cdot \frac{\partial s_0}{\partial W}$$

Recurrent Neural Network

BACKPROPAGATION THROUGH TIME (BPTT)

$$\frac{\partial E_3}{\partial U} = \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial U}$$

$$\frac{\partial s_3}{\partial U} \; ?$$



S_1 and S_2 Depends on W and U too

$$\frac{\partial s_3}{\partial U} \rightarrow \sum_k \frac{\partial s_3}{\partial s_k} \cdot \frac{\partial s_k}{\partial U}$$

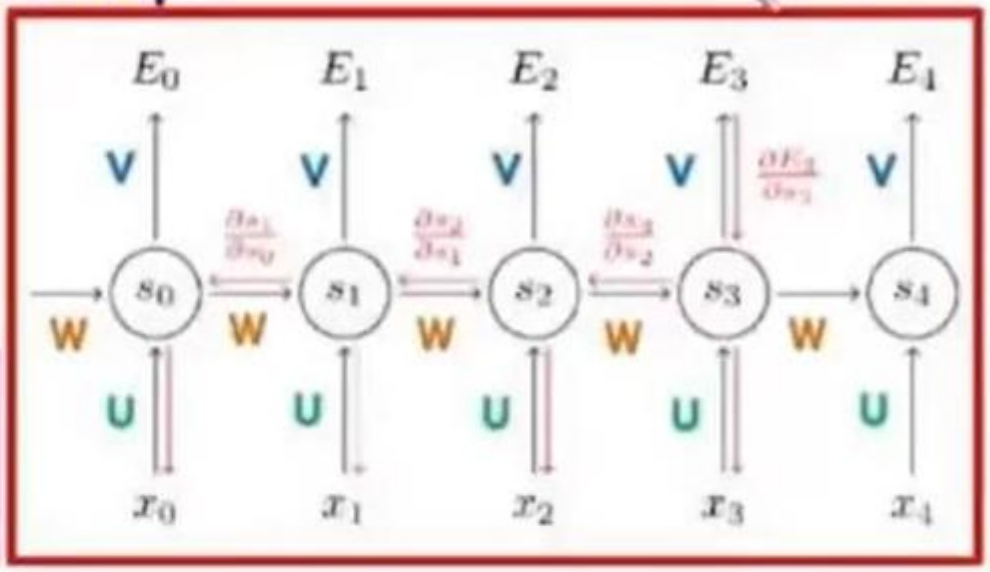
$$\frac{\partial E_3}{\partial U} = \sum_k \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_k} \cdot \frac{\partial s_k}{\partial U}$$

$$s_3 = \tanh(Ws_2 + Ux_3)$$
$$s_2 = \tanh(Ws_1 + Ux_2)$$
$$s_1 = \tanh(Ws_0 + Ux_1)$$

$$\frac{\partial E_3}{\partial U} = \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial U} + \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_2} \cdot \frac{\partial s_2}{\partial U} + \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_1} \cdot \frac{\partial s_1}{\partial U} + \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_0} \cdot \frac{\partial s_0}{\partial U}$$

Recurrent Neural Network

Sum up the gradients at each time step



$$\frac{\partial E}{\partial V} = \sum_t \frac{\partial E_t}{\partial V} = \sum_t \frac{\partial E_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial o_t} \cdot \frac{\partial o_t}{\partial V}$$

$$\frac{\partial E}{\partial W} = \sum_t \frac{\partial E_t}{\partial W} = \sum_t \sum_k \frac{\partial E_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial s_t} \cdot \frac{\partial s_t}{\partial s_k} \cdot \frac{\partial s_k}{\partial W}$$

$$\frac{\partial E}{\partial U} = \sum_t \frac{\partial E_t}{\partial U} = \sum_t \sum_k \frac{\partial E_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial s_t} \cdot \frac{\partial s_t}{\partial s_k} \cdot \frac{\partial s_k}{\partial U}$$

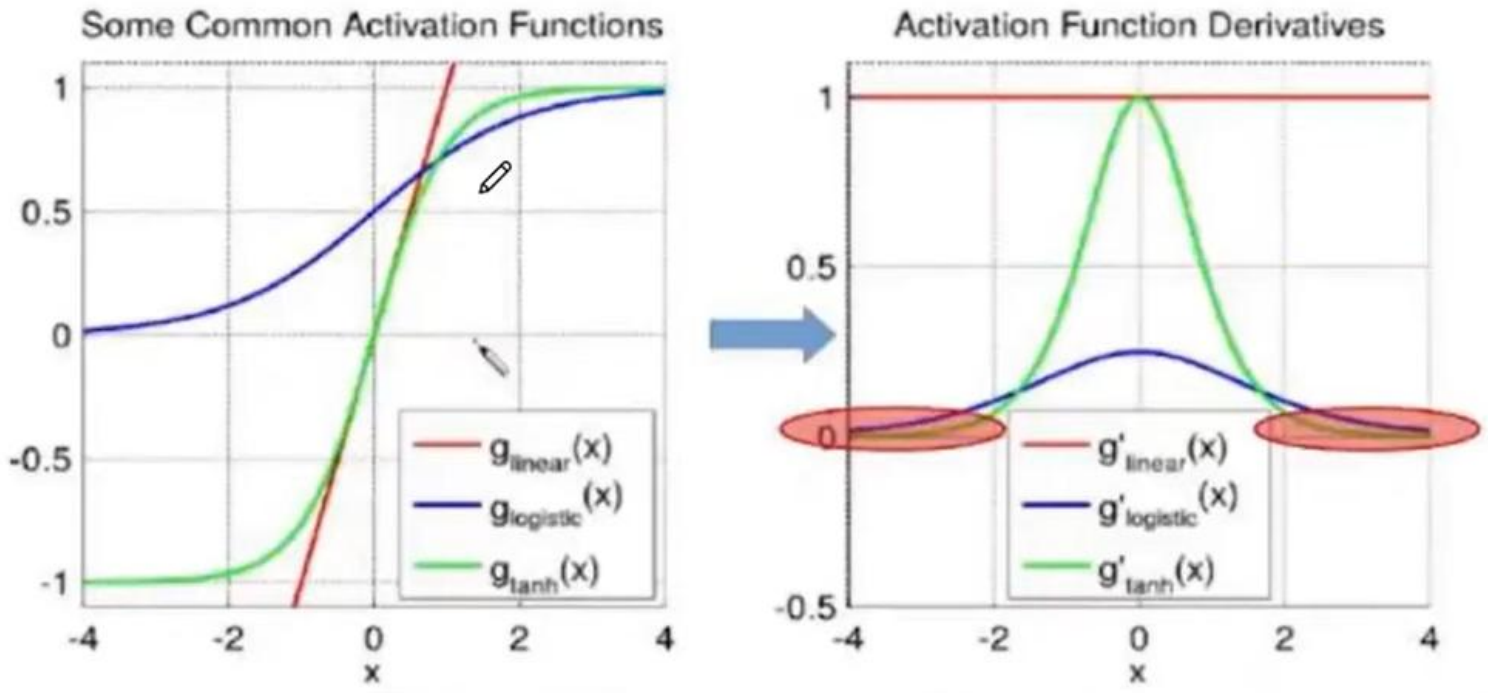
Propagation through time (RNN)

=

Propagation through layers (FNN)

Gradients of Some Common Activation functions

Vanishing gradients



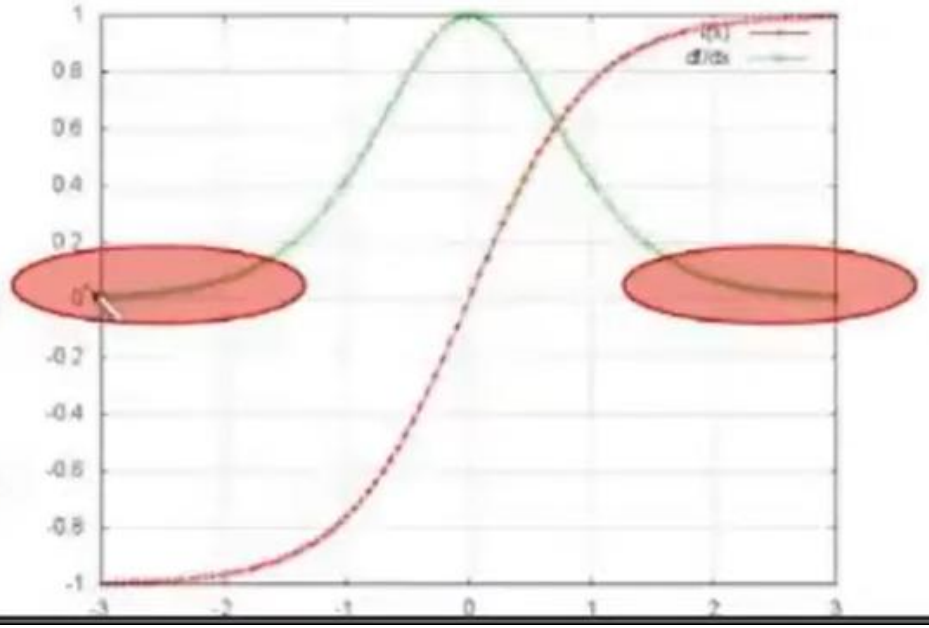
VANISHING GRADIENT PROBLEM

Error Gradients pass through nonlinearity every step

Saturation at both ends ==> zero gradient

Vanishing completely after a few time steps.

tanh and derivative



$$\frac{\partial s_t}{\partial s_{t-1}} = \frac{\partial}{\partial s_{t-1}} \tanh(W s_{t-1} + U x_t)$$

Tanh Derivative ranges from 0 to 1

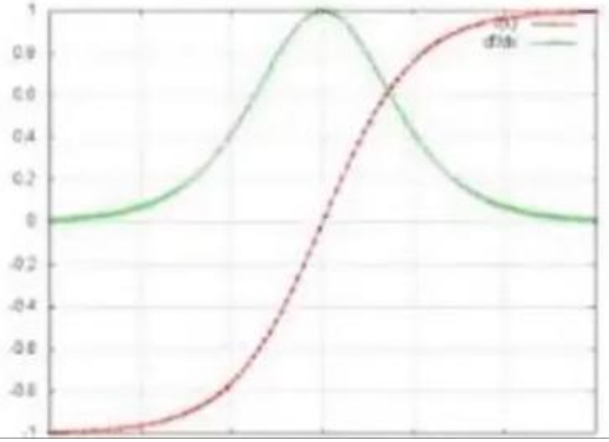
VANISHING GRADIENT PROBLEM

- $$\frac{\partial \mathcal{L}}{\partial c_t} = \frac{\partial \mathcal{L}}{\partial c_T} \cdot \underbrace{\frac{\partial c_T}{\partial c_{T-1}}}_{\rho < 1} \cdot \underbrace{\frac{\partial c_{T-1}}{\partial c_{T-2}}}_{\rho < 1} \cdot \ldots \cdot \underbrace{\frac{\partial c_{t+1}}{\partial c_{c_t}}}_{\rho < 1} \quad \frac{\partial \mathcal{L}}{\partial c_t} \ll 1 \Rightarrow \text{Vanishing gradient}$$
- $$\frac{\partial \mathcal{L}}{\partial c_t} = \frac{\partial \mathcal{L}}{\partial c_T} \cdot \underbrace{\frac{\partial c_T}{\partial c_{T-1}}}_{\rho > 1} \cdot \underbrace{\frac{\partial c_{T-1}}{\partial c_{T-2}}}_{\rho > 1} \cdot \ldots \cdot \underbrace{\frac{\partial c_{t+1}}{\partial c_t}}_{\rho > 1} \quad \frac{\partial \mathcal{L}}{\partial c_t} \gg 1 \Rightarrow \text{Exploding gradient}$$

$$\frac{\partial s_t}{\partial s_{t-1}} = \frac{\partial}{\partial s_{t-1}} \tanh(\mathbf{W}\boldsymbol{s}_{t-1} + \mathbf{U}\boldsymbol{x}_t)$$

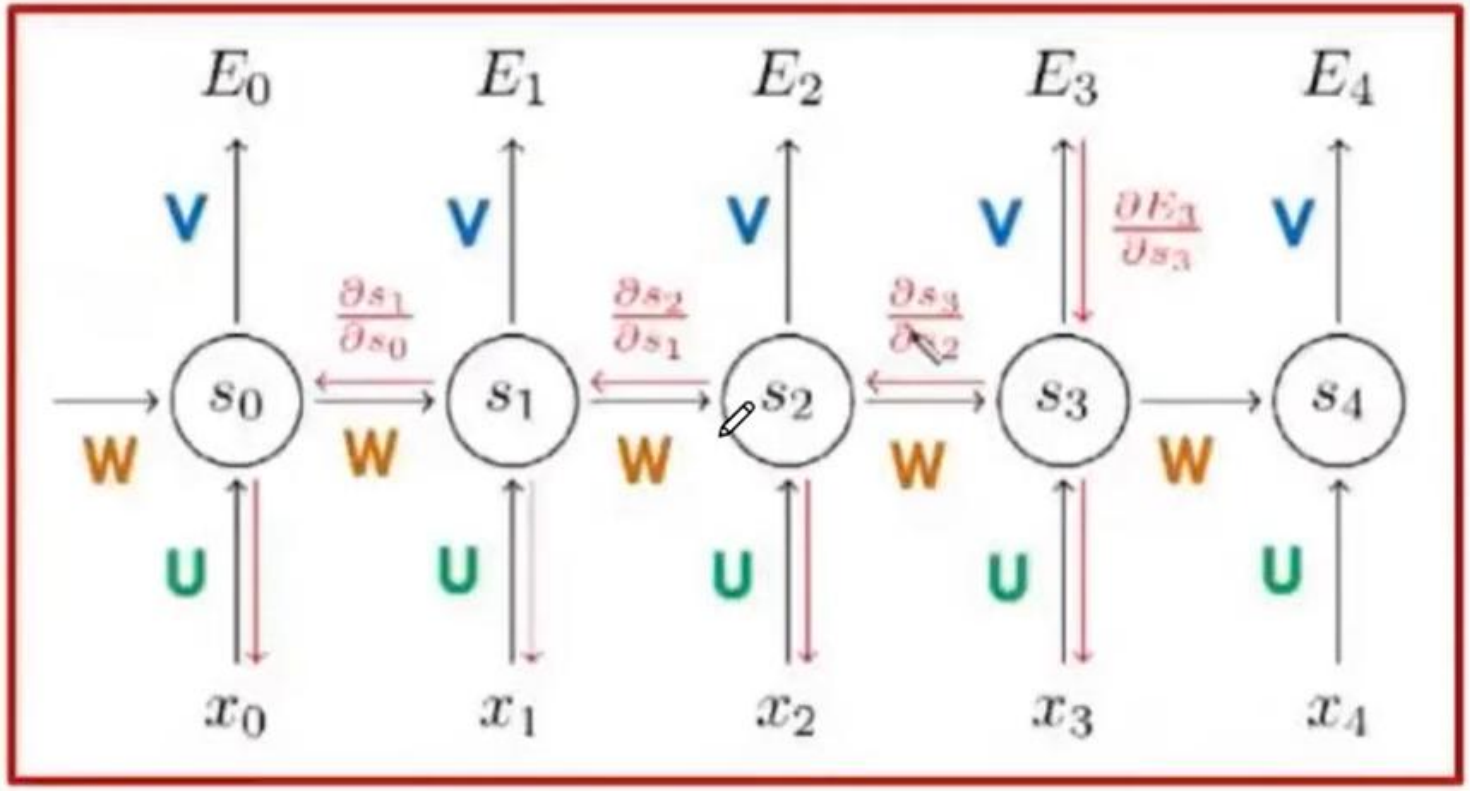
Tanh Derivative ranges from 0 to 1

tanh and derivative



Recurrent Neural Network

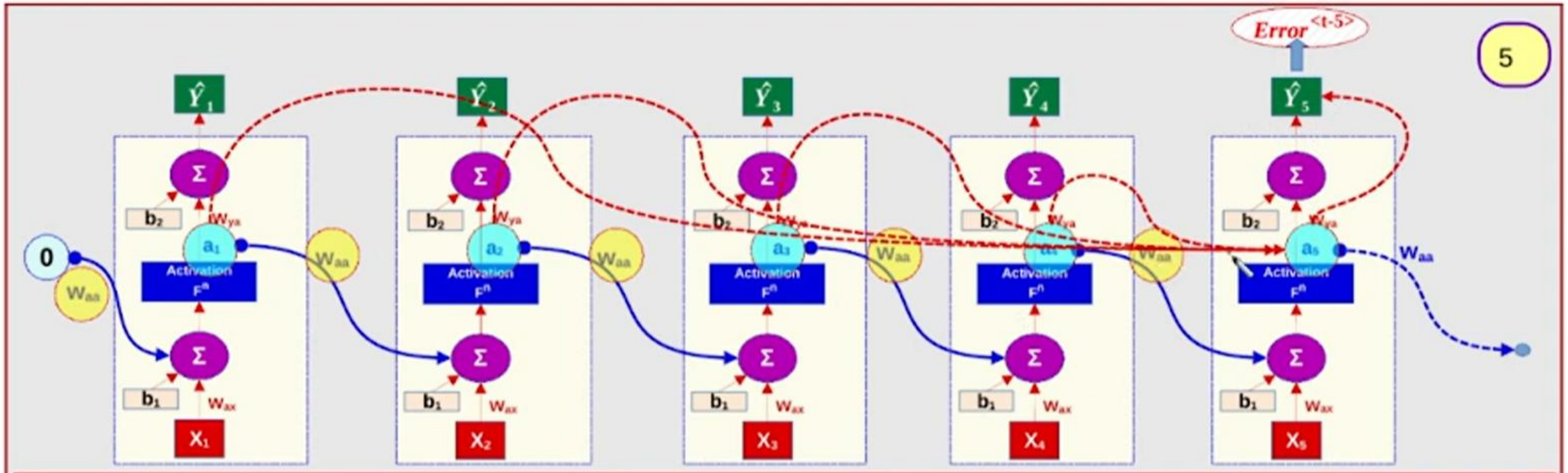
VANISHING GRADIENT PROBLEM



$$\frac{\partial s_t}{\partial s_{t-1}} = \frac{\partial}{\partial s_{t-1}} \tanh(\mathrm{W}\boldsymbol{s}_{t-1} + \mathrm{U}\boldsymbol{x}_t)$$

Tanh Derivative ranges from 0 to 1

From
Recurrent Neural Network (RNN)
to
Lone Short Term Memory (LSTM)



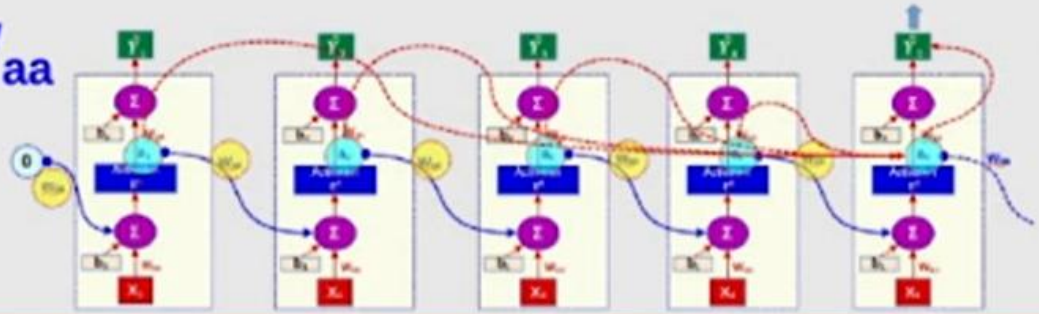
$$\frac{\partial Error_{(\hat{Y}_5)}}{\partial w_{aa}} = \frac{\partial Error_{(\hat{Y}_5)}}{\partial \hat{Y}_5} * \frac{\partial \hat{Y}_5}{\partial w_{aa}} =$$

Remember: $w_{aa}^{updated} = w_{aa}^{old} - \eta * \frac{\partial Error_{(Total)}}{\partial w_{aa}}$

$$= \frac{\partial Error_{(\hat{Y}_5)}}{\partial \hat{Y}_5}\left[\frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_4} * \frac{\partial a_4}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_3} * \frac{\partial a_3}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_2} * \frac{\partial a_2}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_1} * \frac{\partial a_1}{\partial w_{aa}}\right]$$

$$= \frac{\partial Error_{(\hat{Y}_5)}}{\partial \hat{Y}_5}\left[\frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial a_2} * \frac{\partial a_2}{\partial w_{aa}} + \ldots\ldots\right]$$

Gradient of Error at Y_5 due to W_{aa}



$$\frac{\partial Error_{(\hat{Y}_5)}}{\partial w_{aa}} = \frac{\partial Error_{(\hat{Y}_5)}}{\partial \hat{Y}_5} * \frac{\partial \hat{Y}_5}{\partial w_{aa}} =$$

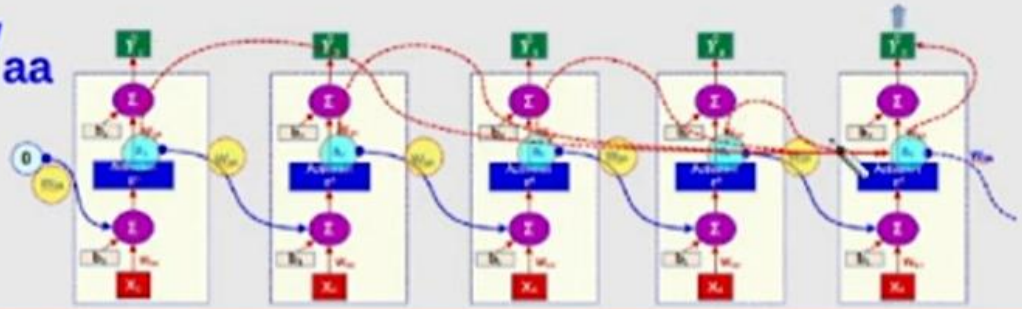
$$= \frac{\partial Error_{(\hat{Y}_5)}}{\partial \hat{Y}_5}\left[\frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_4} * \frac{\partial a_4}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_3} * \frac{\partial a_3}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_2} * \frac{\partial a_2}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_1} * \frac{\partial a_1}{\partial w_{aa}}\right]$$

$$= \frac{\partial Error_{(\hat{Y}_5)}}{\partial \hat{Y}_5}\left[\frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial a_2} * \frac{\partial a_2}{\partial w_{aa}} + \cdots\cdots\right]$$

$$\frac{\partial Error_{(\hat{Y}_5)}}{\partial w_{aa}} = \frac{\partial Error_{(\hat{Y}_5)}}{\partial Y_5} \sum_{k=1}^{5}\left[\frac{\partial \hat{Y}_5}{\partial a_k} * \frac{\partial a_k}{\partial w_{aa}}\right] = \sum_{k=1}^{5} \frac{\partial Error_{(\hat{Y}_5)}}{\partial Y_5} * \frac{\partial \hat{Y}_5}{\partial a_k} * \frac{\partial a_k}{\partial w_{aa}}$$

Move inside the summation

Gradient of Error at Y_5 due to W_{aa}



$$\frac{\partial Error_{(\hat{Y}_5)}}{\partial w_{aa}} = \frac{\partial Error_{(\hat{Y}_5)}}{\partial \hat{Y}_5} * \frac{\partial \hat{Y}_5}{\partial w_{aa}} =$$

$$= \frac{\partial Error_{(\hat{Y}_5)}}{\partial \hat{Y}_5}\left[\frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_4} * \frac{\partial a_4}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_3} * \frac{\partial a_3}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_2} * \frac{\partial a_2}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_1} * \frac{\partial a_1}{\partial w_{aa}}\right]$$

$$= \frac{\partial Error_{(\hat{Y}_5)}}{\partial \hat{Y}_5}\left[\frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial a_2} * \frac{\partial a_2}{\partial w_{aa}} + \cdots\cdots\right]$$

$$\frac{\partial Error_{(\hat{Y}_5)}}{\partial w_{aa}} = \frac{\partial Error_{(\hat{Y}_5)}}{\partial Y_5} \sum_{k=1}^{5}\left[\frac{\partial \hat{Y}_5}{\partial a_k} * \frac{\partial a_k}{\partial w_{aa}}\right] = \sum_{k=1}^{5} \frac{\partial Error_{(\hat{Y}_5)}}{\partial Y_5} * \frac{\partial \hat{Y}_5}{\partial a_k} * \frac{\partial a_k}{\partial w_{aa}} = \sum_{k=1}^{5} \frac{\partial Error_{(\hat{Y}_5)}}{\partial Y_5} * \frac{\partial a_k}{\partial w_{aa}} * \frac{\partial \hat{Y}_5}{\partial a_5} * \prod_{j=k}^{5-1} \frac{\partial a_{j+1}}{\partial a_j}$$

In General $Error_t$ (due to Y_t ONLY) $\Rightarrow$

t=5 in the previous derivation

$$\frac{\partial Error_t}{\partial w_{aa}} = \frac{\partial Error_t}{\partial Y_t} \sum_{k=1}^{t}\left[\frac{\partial \hat{Y}_t}{\partial a_k} * \frac{\partial a_k}{\partial w_{aa}}\right] = \sum_{k=1}^{t} \frac{\partial Error_t}{\partial Y_t} * \frac{\partial a_k}{\partial w_{aa}} * \frac{\partial \hat{Y}_t}{\partial a_t} * \prod_{j=k}^{t-1} \frac{\partial a_{j+1}}{\partial a_j}$$

Gradient of Total Error due to W_{aa}

In General: $Error_t$ **(due to Y_t ONLY)**

$$\frac{\partial Error_t}{\partial w_{aa}} = \frac{\partial Error_t}{\partial Y_t} \sum_{k=1}^{t} \left[\frac{\partial \hat{Y}_t}{\partial a_k} * \frac{\partial a_k}{\partial w_{aa}} \right] = \sum_{k=1}^{t} \frac{\partial Error_t}{\partial Y_t} * \frac{\partial a_k}{\partial w_{aa}} * \frac{\partial \hat{Y}_t}{\partial a_t} * \prod_{j=k}^{t-1} \frac{\partial a_{j+1}}{\partial a_j}$$

Total Error $(at\ all\ time\ steps)$ due to changes in W_{aa}

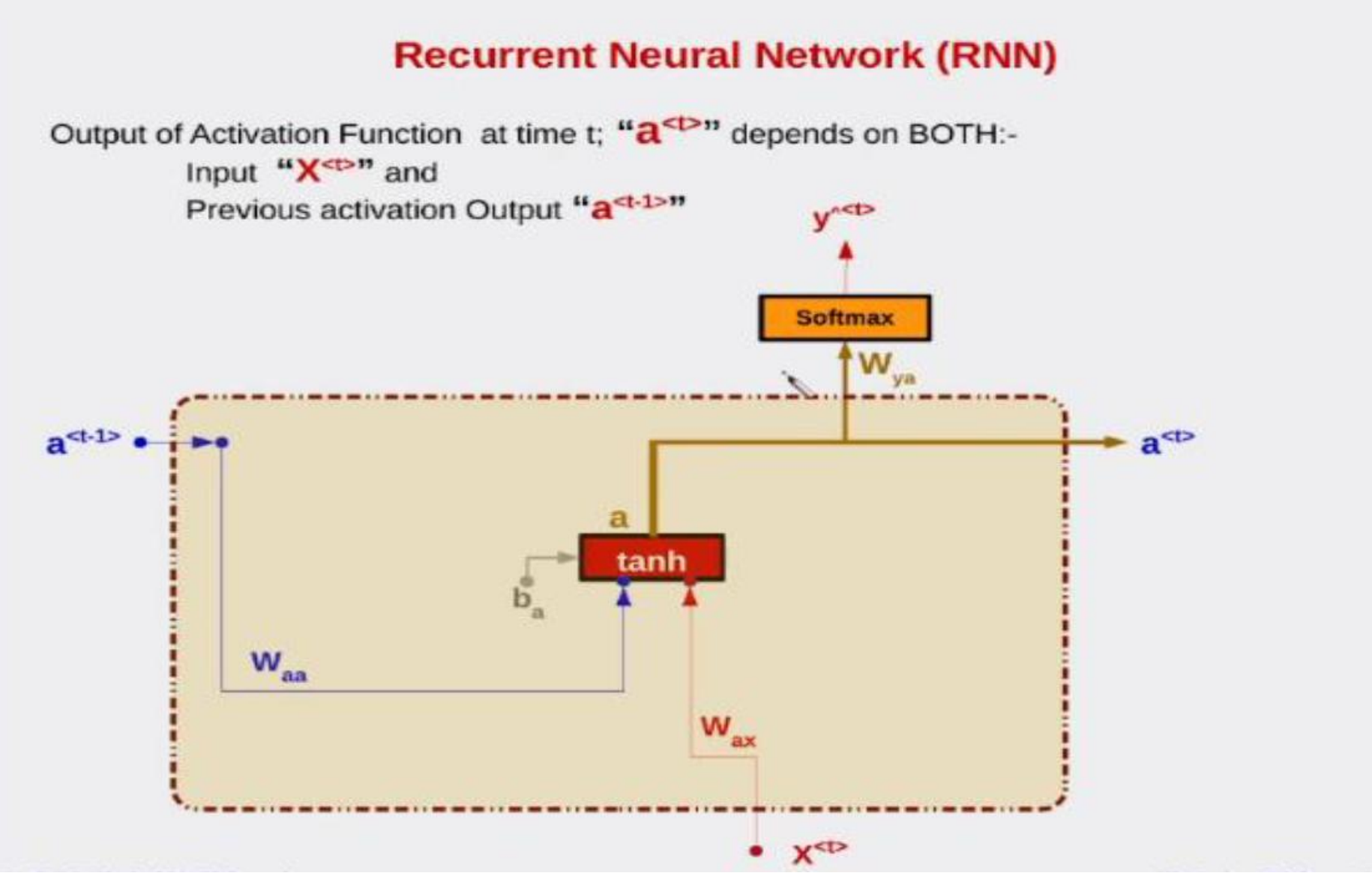
$$\frac{\partial Error_{(total)}}{\partial W_{aa}} = \sum_{t=1}^{\text{end of Seq T}} \frac{\partial Error_{(t)}}{\partial W_{aa}} = \sum_{t=1}^{\text{end of Seq T}} \left[\sum_{k=1}^{t} \frac{\partial Error_t}{\partial Y_t} * \frac{\partial a_k}{\partial w_{aa}} * \frac{\partial \hat{Y}_t}{\partial a_t} * \boxed{\prod_{j=k}^{t-1} \frac{\partial a_{j+1}}{\partial a_j}} \right]$$

$a_{j+1} = \tanh(a_j * w_{aa} + x_{j+1} * w_{ax} + bias)$ $\Rightarrow$ value of $\frac{\partial a_{j+1}}{\partial a_j}$ may be in range $[0 \ldots 1]$ $\Rightarrow$ ***Fraction***

for $T = 100$ timesteps, the longest length of fractions multiplication = T - 1 = 99 times $\Rightarrow$ ***Vanish***

Remember: $\tanh(z)$ $\Rightarrow$ $[-1..1]$ $\frac{\partial \tanh(z)}{\partial(z)} = 1 - \tanh^2 z$ $\Rightarrow$ ***Min value = 0*** (at tanh(z) = 1 or -1) ***Max vlaue = 1*** $(at\ \tanh(z) = 0)$

Recurrent Neural Network



Recurrent Neural Network

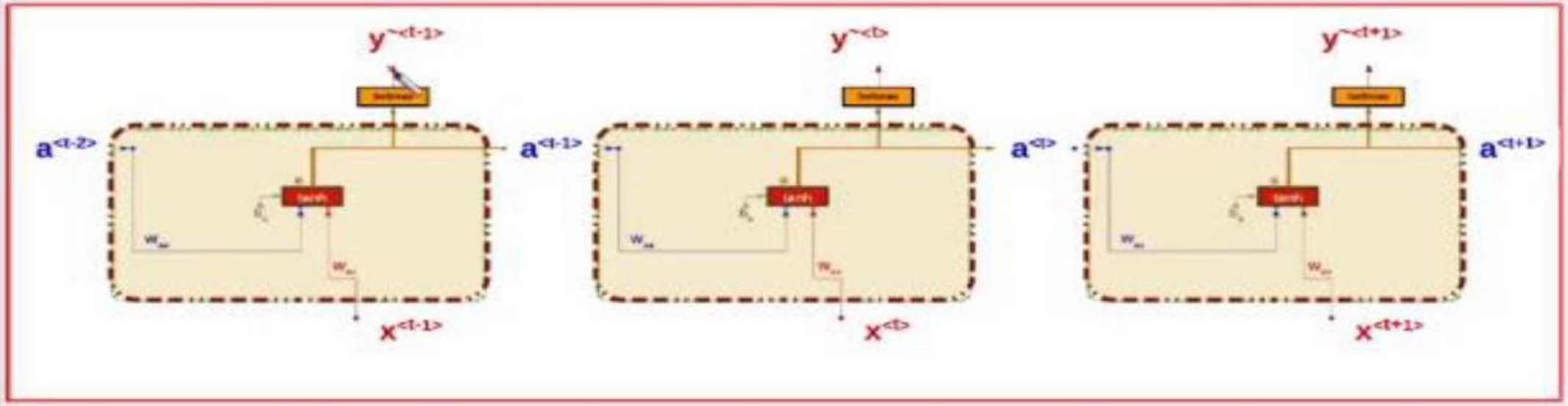
Recurrent Neural Network (RNN)

Output of Activation Function at time t; "$a^{<t>}$" depends on BOTH:-

Input "$X^{<t>}$" and

Previous activation Output "$a^{<t-1>}$"

Inputs at time t-1 , t , t+1



Recurrent Neural Network

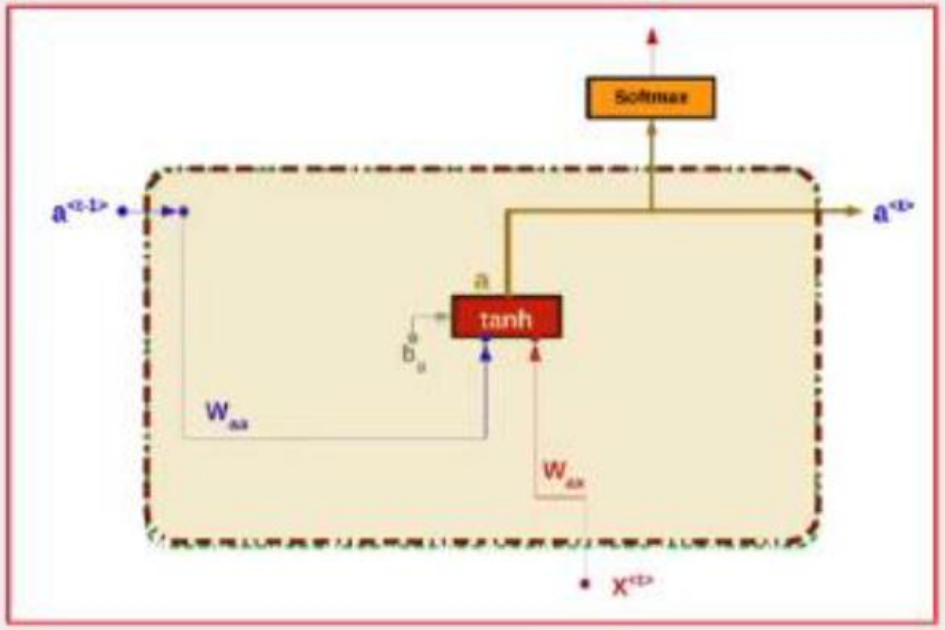
Recurrent Neural Network (RNN)

From Prev. Current I/P

$$a^{<1>} = F^n\,(W_{aa}a^{<0>} + W_{ax}x^{<1>} + b_a)$$

$$y\hat{}^{<1>} = F^n\,(W_{ya}\,a^{<1>} + b_y)$$

Non Linearity



Recurrent Neural Network

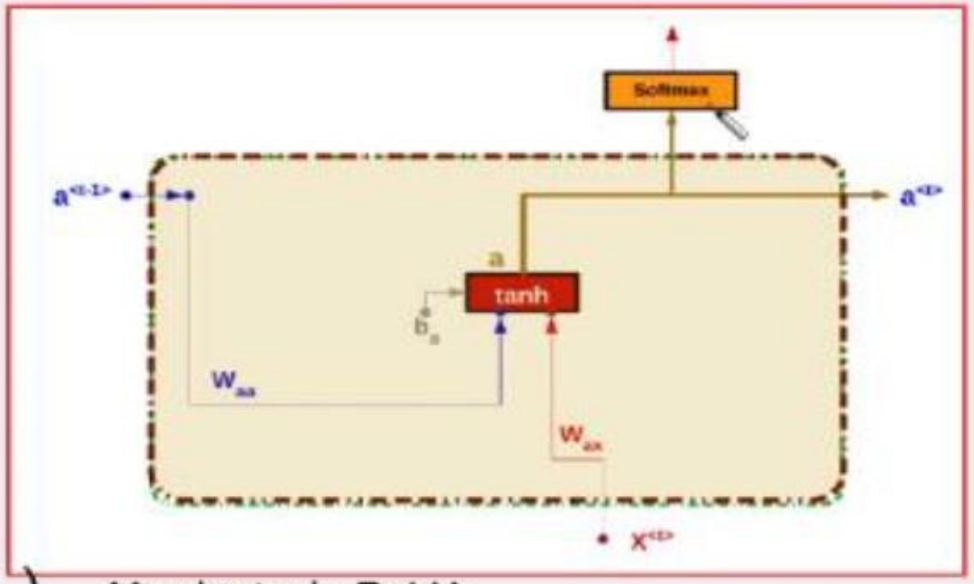
Recurrent Neural Network (RNN)

From Prev. Current I/P

$$a^{<1>} = F^n\,(W_{aa}a^{<0>} + W_{ax}x^{<1>} + b_a)$$

$$y\hat{}^{<1>} = F^n\,(W_{ya}\,a^{<1>} + b_y)$$

Non Linearity



$$a^{<1>} = \tanh\,(W_{aa}a^{<0>} + W_{ax}x^{<1>} + b_a)$$ May be tanh, ReLU, ...

$$y\hat{}^{<1>} = \text{Softmax}\,(W_{ya}\,a^{<1>} + b_y)$$ Segmoid for Binary O/P, Softmax for Multi-Class O/P

$$a^{<t>} = \tanh\,(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a)$$ May be tanh, ReLU, ...

$$y\hat{}^{<t>} = \text{Softmax}\,(W_{ya}\,a^{<t>} + b_y)$$ Segmoid for Binary O/P, Softmax for Multi-Class O/P

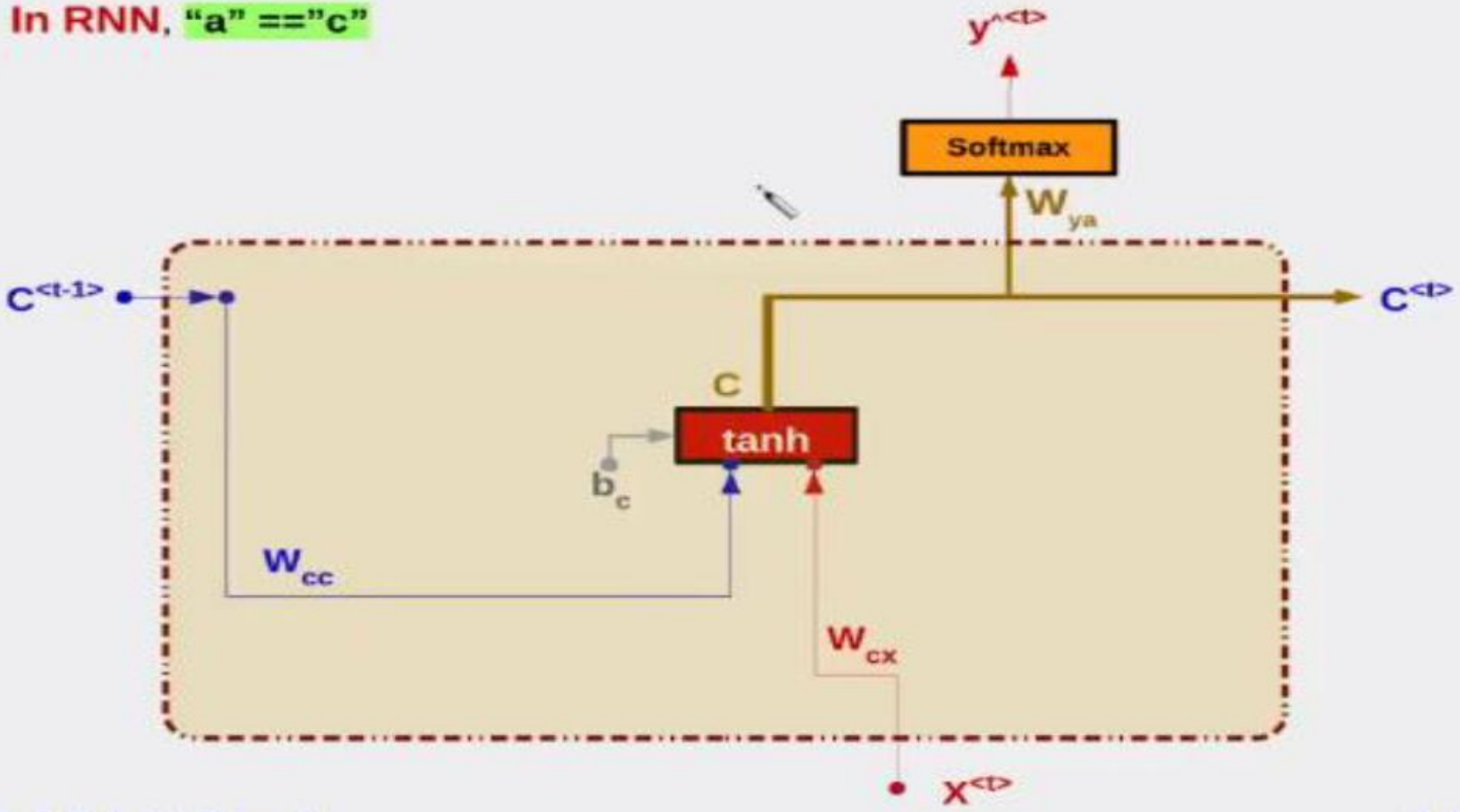
[2] Gated Recurrent Unit (GRU)

Recurrent Neural Network

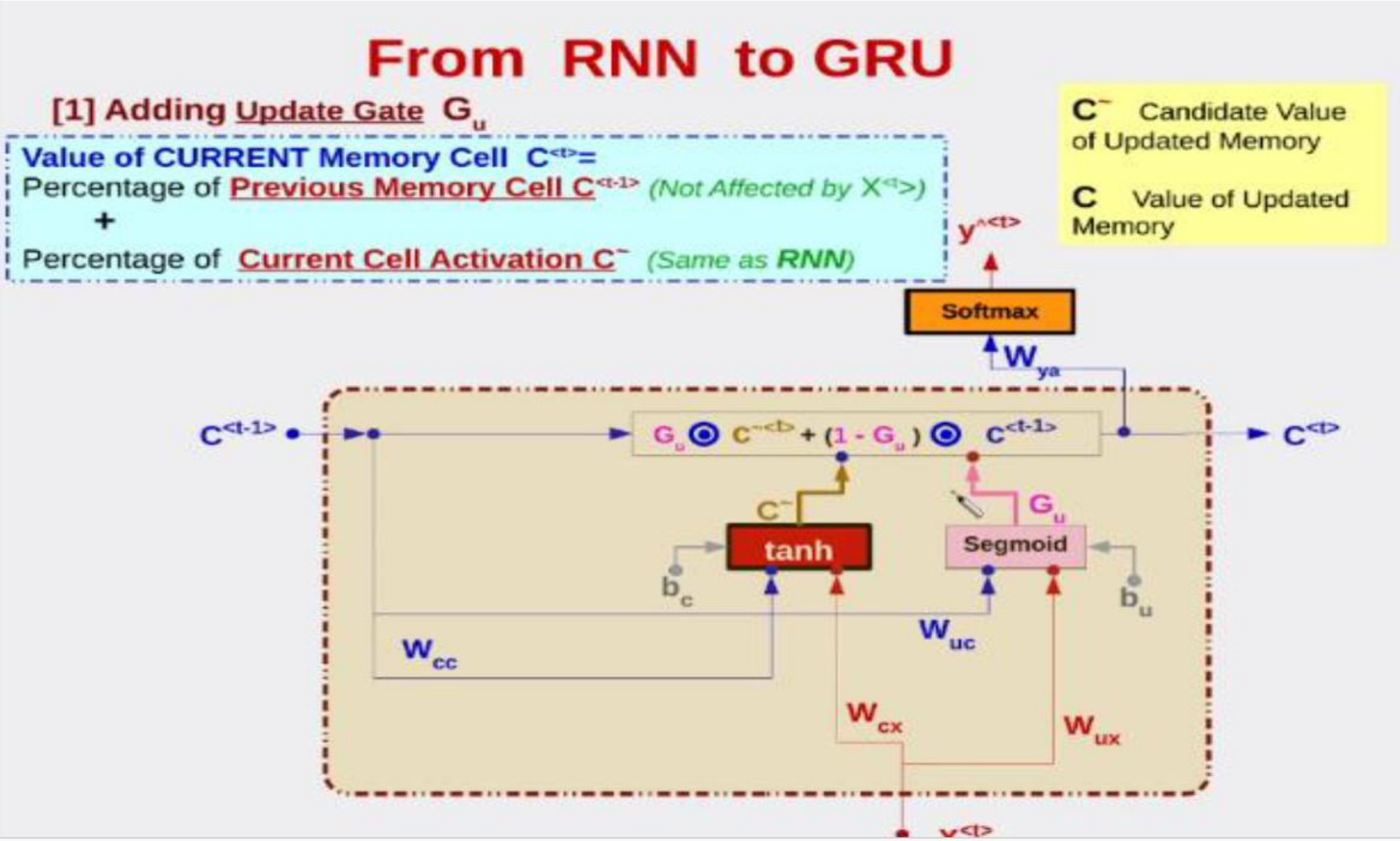
From RNN to GRU

Define **Memory** Cell "**C**" in addition to Output of **Activation** function "**a**".

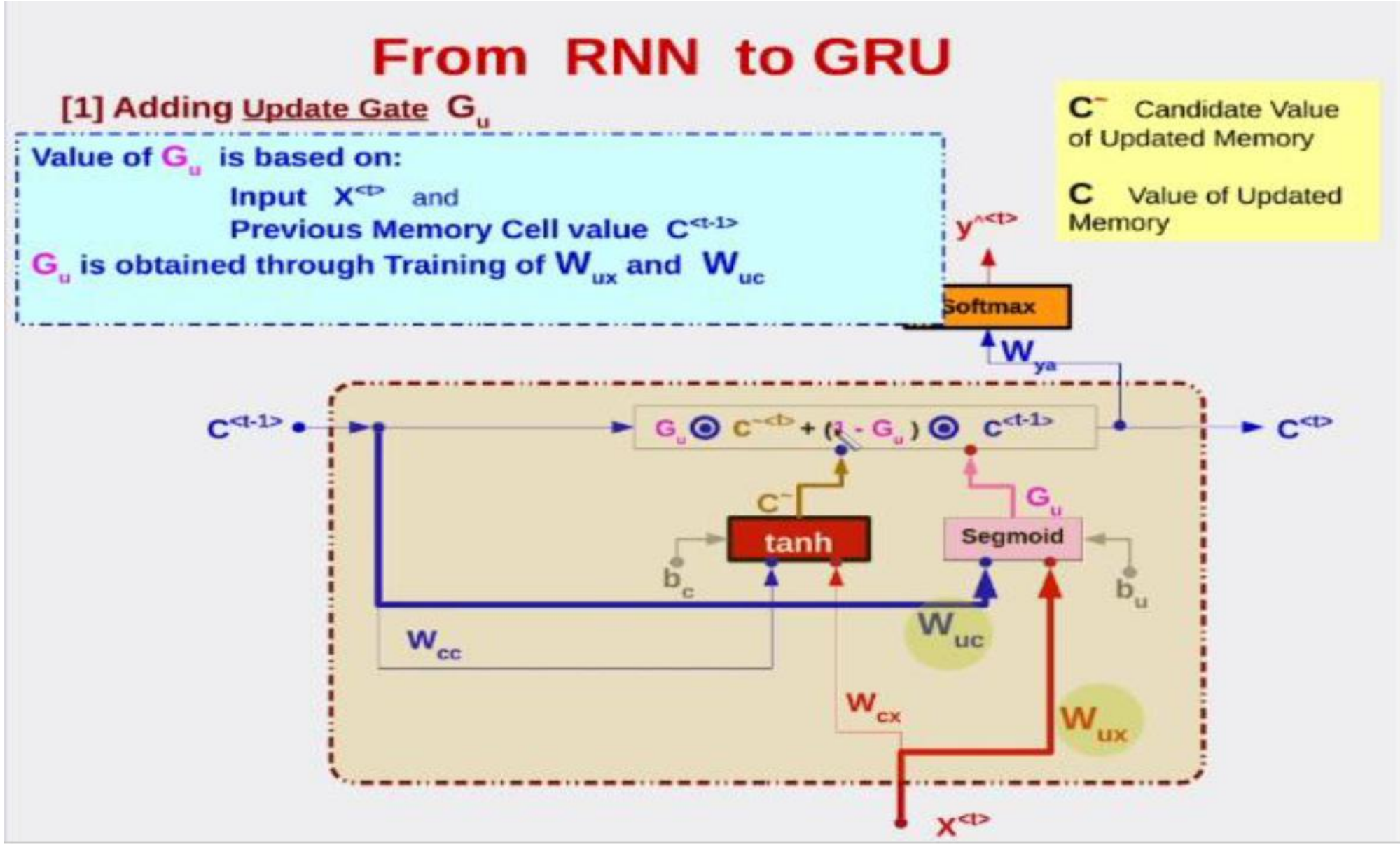
In RNN, "a" =="c"



Recurrent Neural Network



Recurrent Neural Network



Recurrent Neural Network

From RNN to GRU

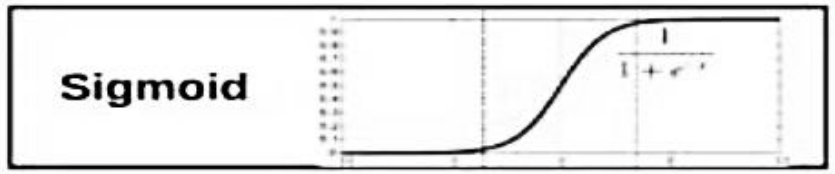
[1] Adding Update Gate G_u

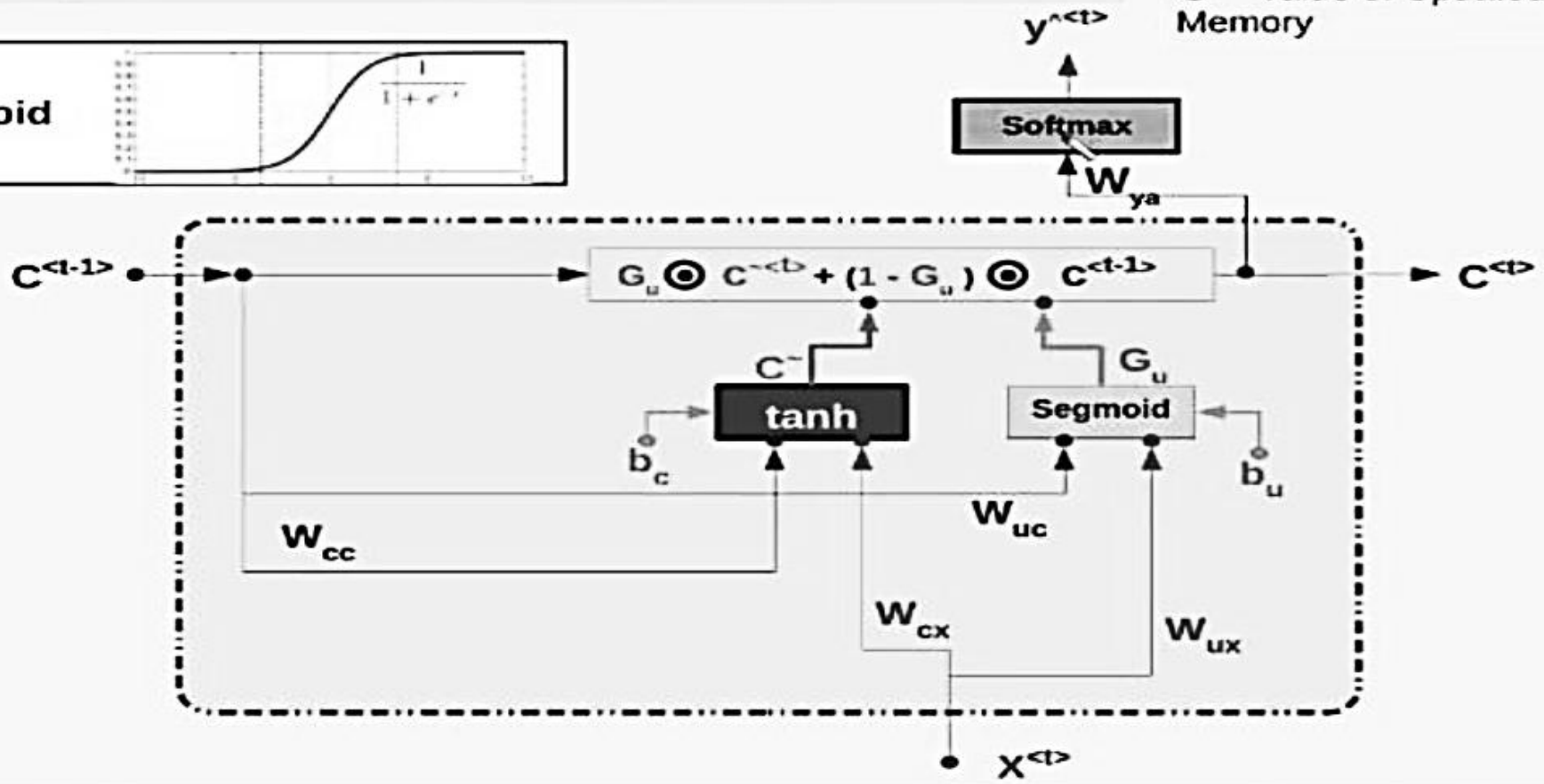
If $G_u = 0$, Keep Memory Value "$C^{<t>}$" Same as Previous Value "$C^{<t-1>}$"

If $G_u = 1$, Forget Previous Memory Value "$C^{<t-1>}$"

$C^{\sim}$ Candidate Value of Updated Memory

C Value of Updated Memory





Recurrent Neural Network

From RNN to GRU

$$G_u = \text{Sigmoid}\,(W_{uc}c^{<t-1>} + W_{ux}x^{<t>} + b_u)$$

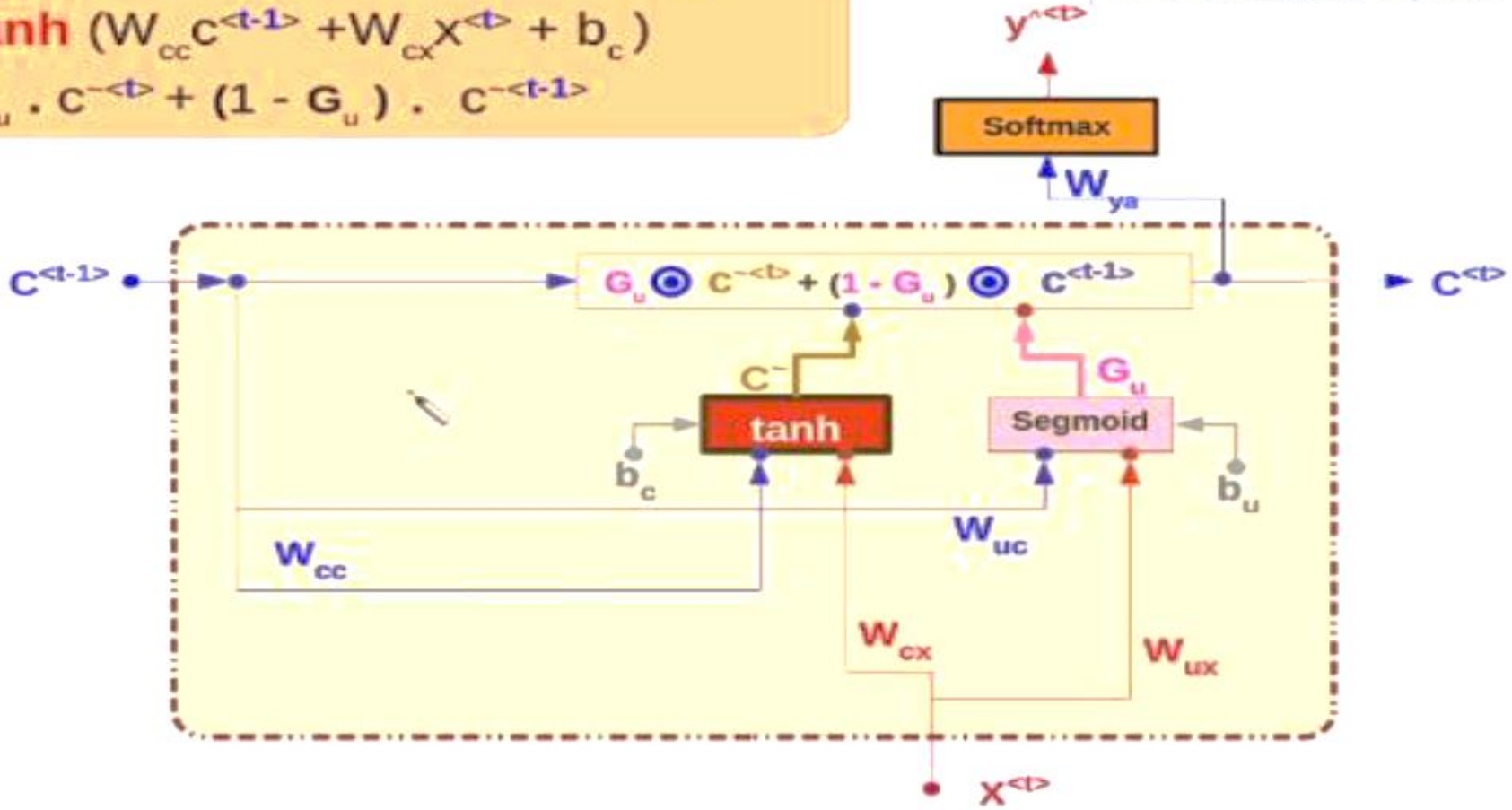
$$c^{\sim<t>} = \tanh\,(W_{cc}c^{<t-1>} + W_{cx}x^{<t>} + b_c)$$

$$c^{<t>} = G_u \,.\, c^{\sim<t>} + (1 - G_u)\,.\,c^{\sim<t-1>}$$

$C^{\sim}$ is the Candidate Update

G_u is the Update Gate

C is the Actual Update

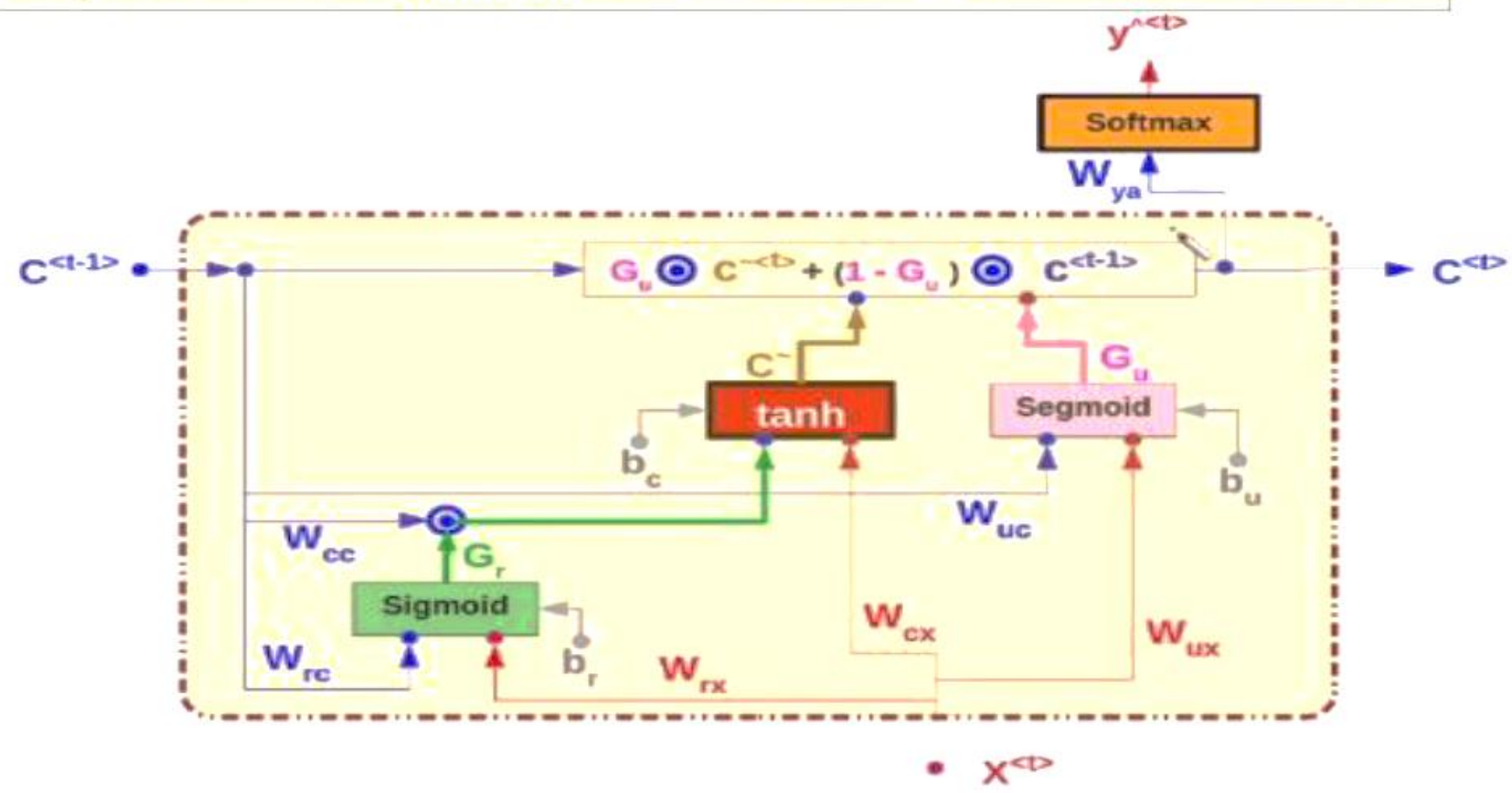


Recurrent Neural Network

From RNN to GRU

[2] Adding Relevance Gate G_r

If $G_r = 1$, $C^{<t-1>}$ is Relevant to update Candidate Memory cell value "$C^{\sim}$"
If $G_r = 0$, $C^{<t-1>}$ is IrRelevant to update Candidate Memory cell value "$C^{\sim}$"



Recurrent Neural Network

From RNN to GRU

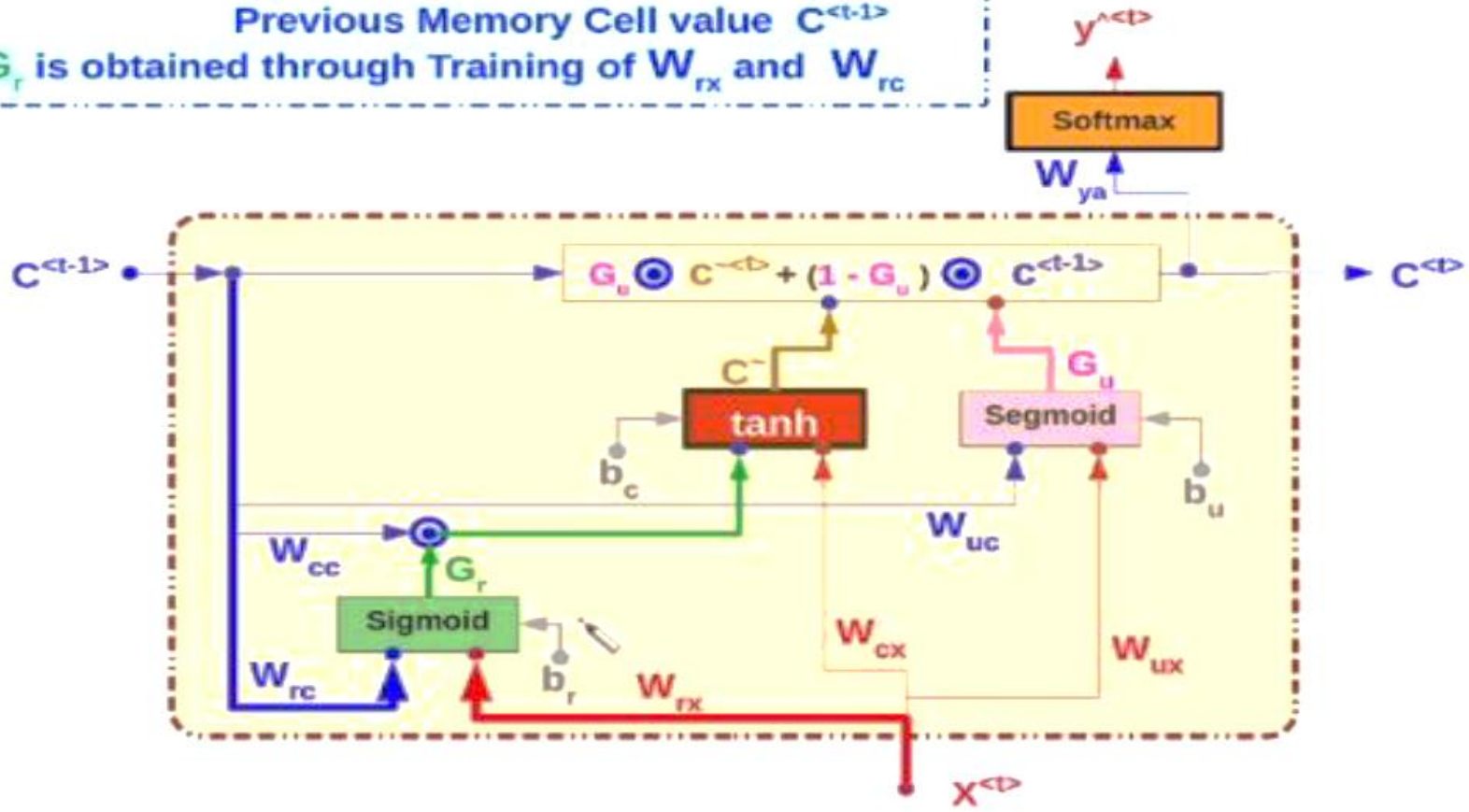
[2] Adding Relevance Gate G_r

Value of G_r is based on:

Input $X^{<t>}$ and

Previous Memory Cell value $C^{<t-1>}$

G_r is obtained through Training of W_{rx} and W_{rc}



Recurrent Neural Network

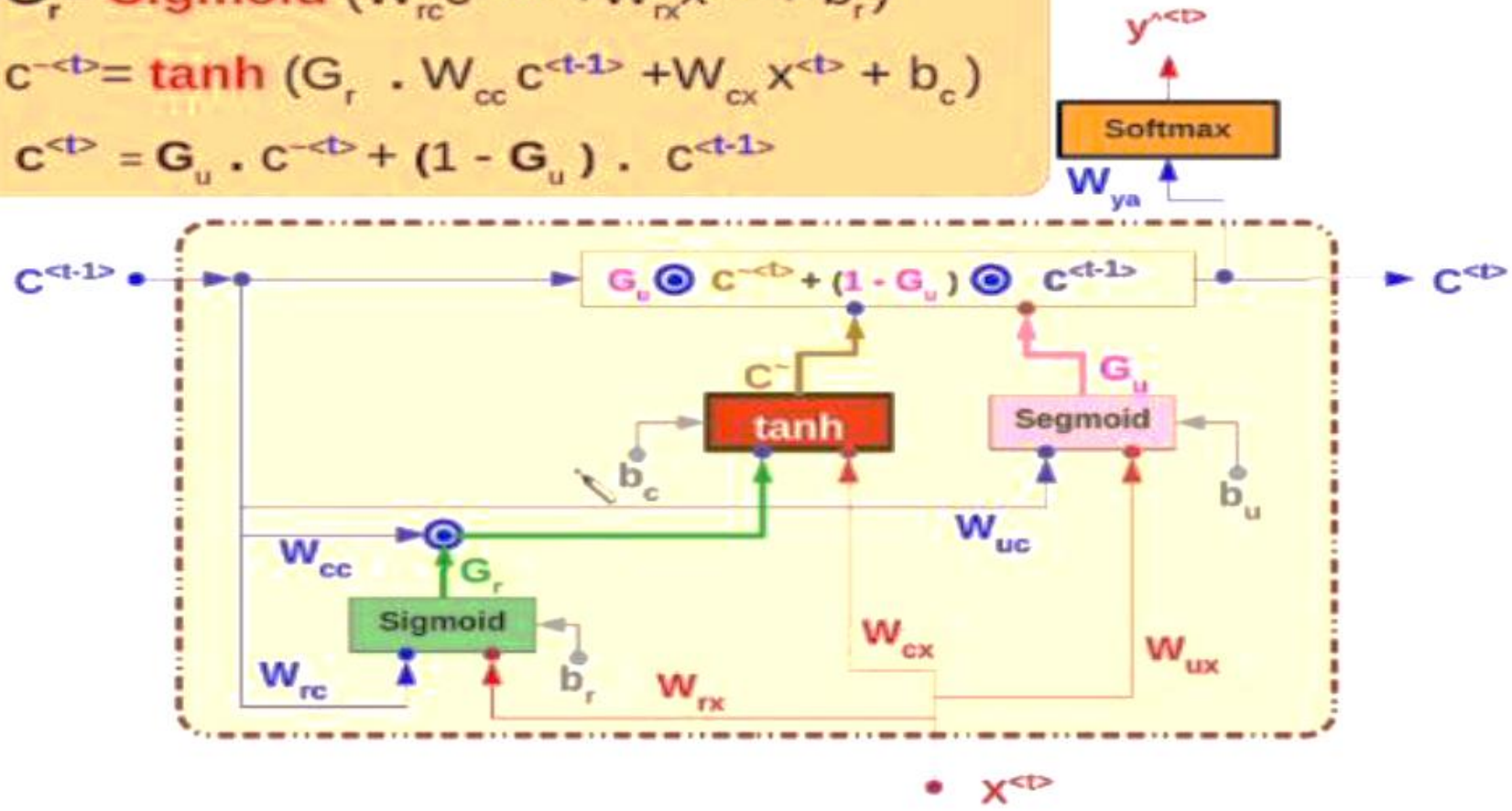
From RNN to GRU

$$G_u = \text{Sigmoid}\,(W_{uc} c^{<t-1>} + W_{ux} x^{<t>} + b_u)$$

$$G_r = \text{Sigmoid}\,(W_{rc} c^{<t-1>} + W_{rx} x^{<t>} + b_r)$$

$$c^{\sim<t>} = \tanh\,(G_r \,.\, W_{cc}\, c^{<t-1>} + W_{cx} x^{<t>} + b_c)$$

$$c^{<t>} = G_u \,.\, c^{\sim<t>} + (1 - G_u) \,.\, c^{<t-1>}$$



Recurrent Neural Network

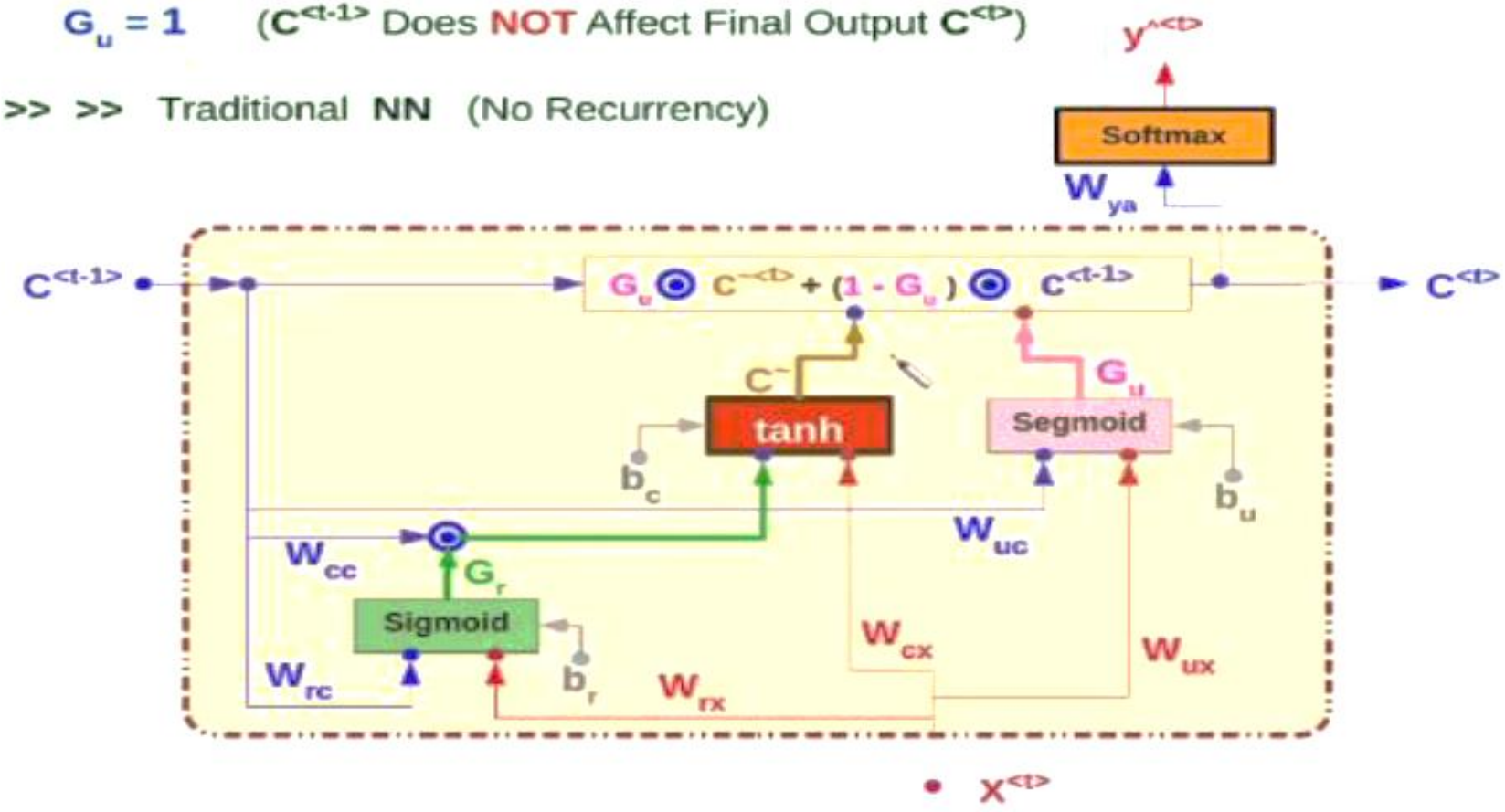
From RNN to GRU

IF

$G_r = 0$ ($C^{<t-1>}$ Does NOT Affect tanh Output $C^{\sim}$)

$G_u = 1$ ($C^{<t-1>}$ Does NOT Affect Final Output $C^{<t>}$)

>> >> Traditional NN (No Recurrency)



Recurrent Neural Network

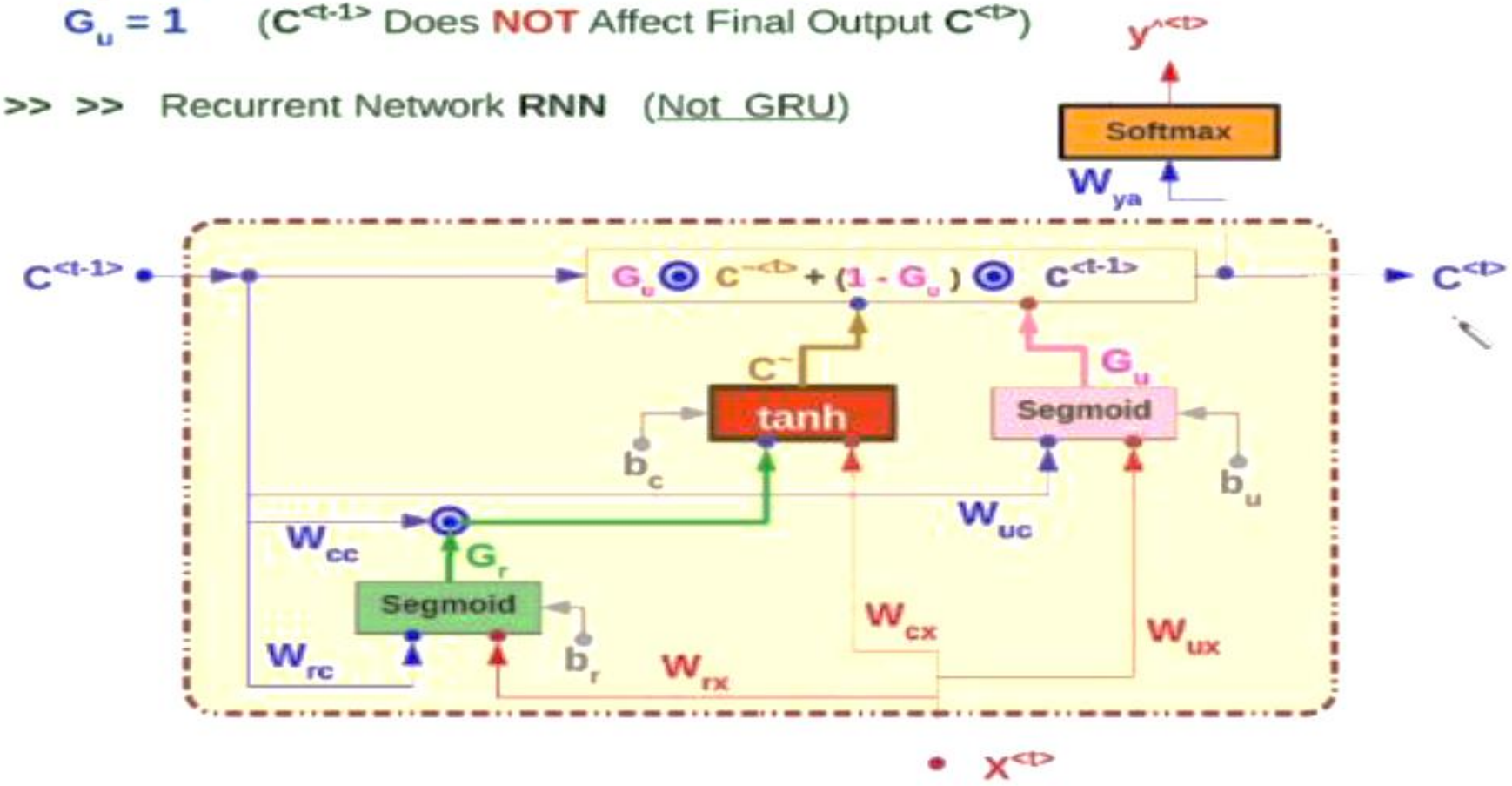
From RNN to GRU

IF

$G_r = 1$ ($C^{<t-1>}$ **Affects** tanh Output $C^{\sim}$)

$G_u = 1$ ($C^{<t-1>}$ Does **NOT** Affect Final Output $C^{<t>}$)

>> >> Recurrent Network **RNN** (Not GRU)



Recurrent Neural Network

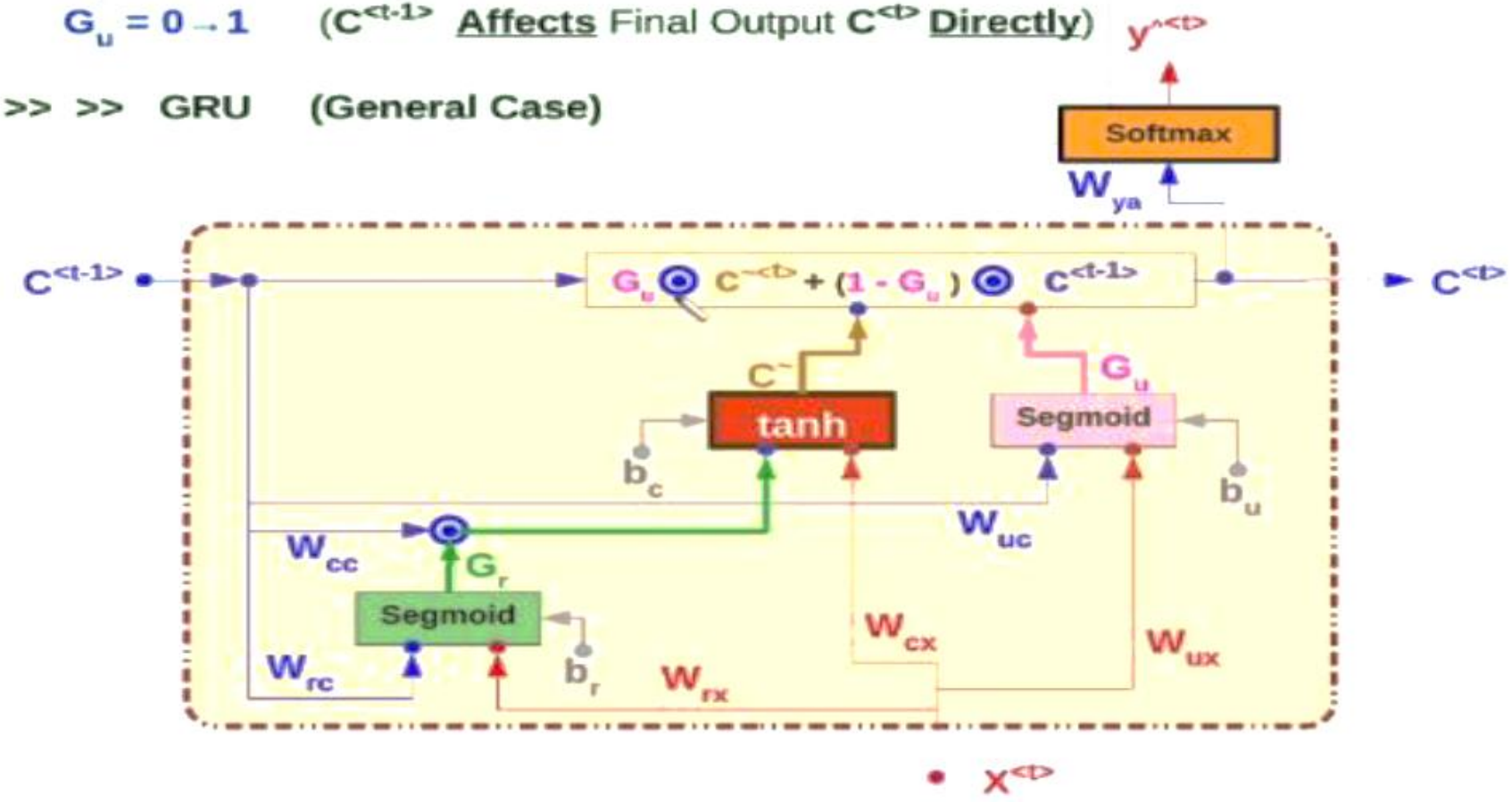
From RNN to GRU

IF

$G_r = 0 \rightarrow 1$ ($C^{<t-1>}$ **Affects** tanh Output $C^{\sim}$)

$G_u = 0 \rightarrow 1$ ($C^{<t-1>}$ **Affects** Final Output $C^{<t>}$ **Directly**)

>> >> GRU (General Case)



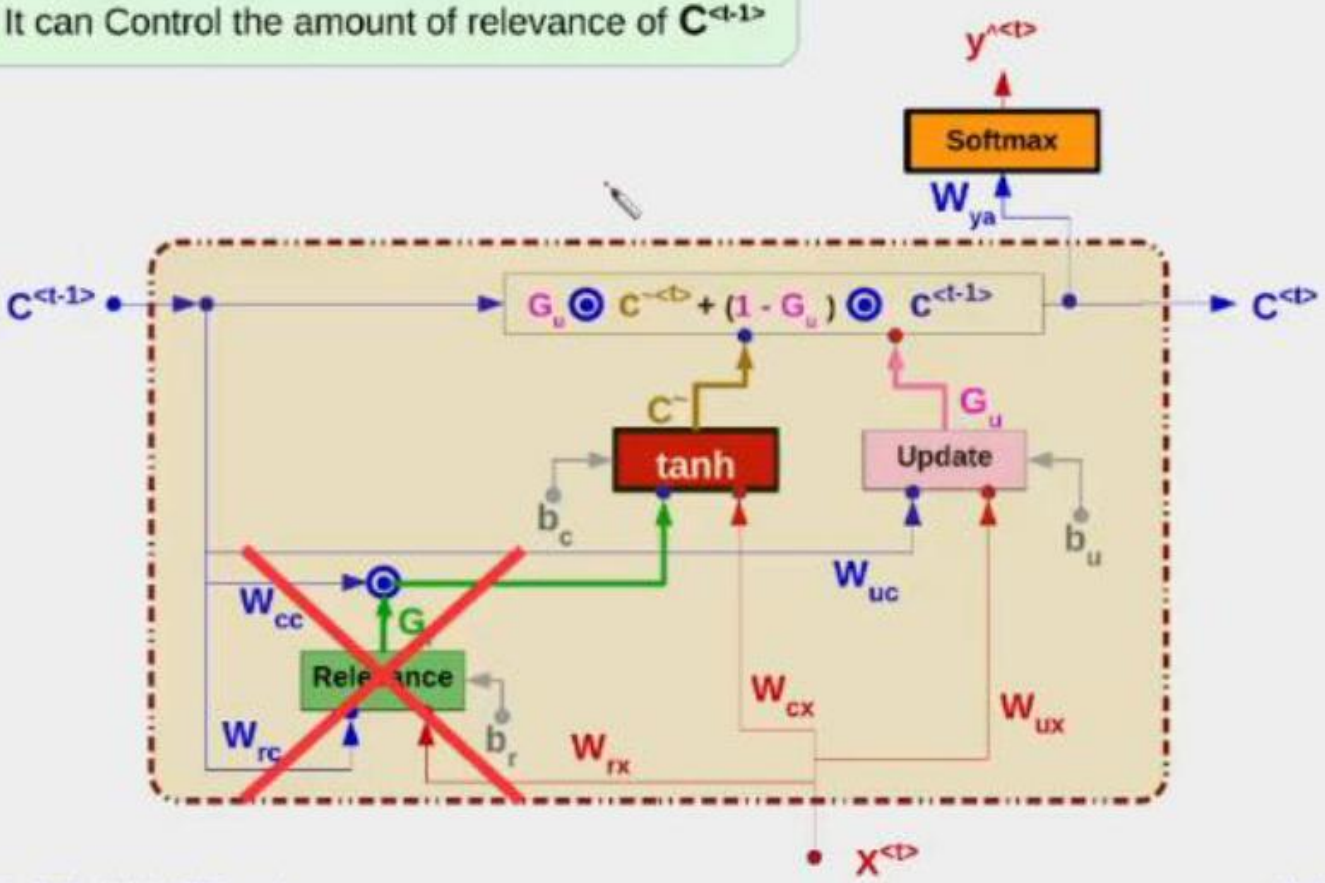
[3] Long Short Term Memory (LSTM)

Recurrent Neural Network

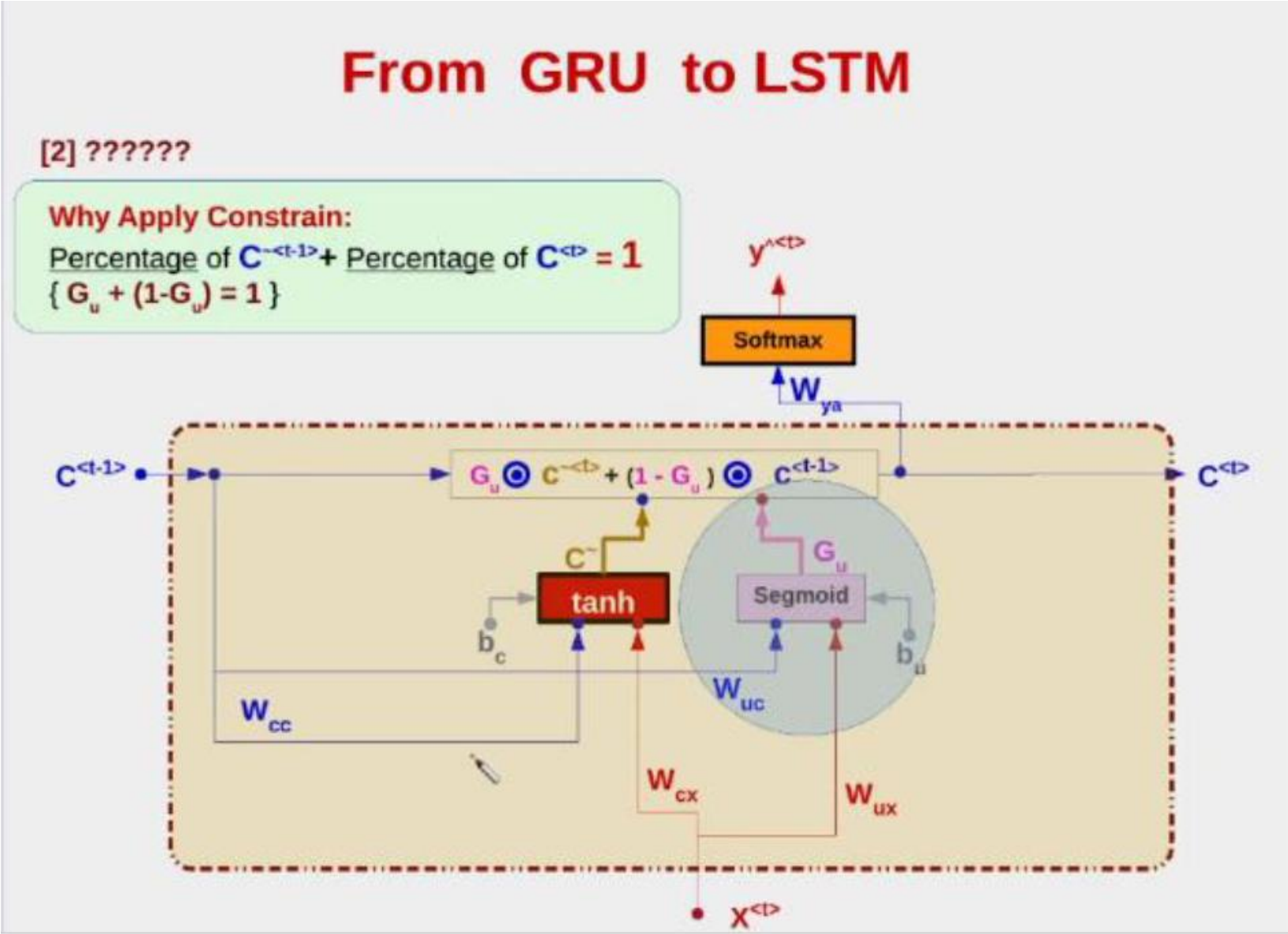
From GRU to LSTM

[1] Removing **Relevance Gate** G_r

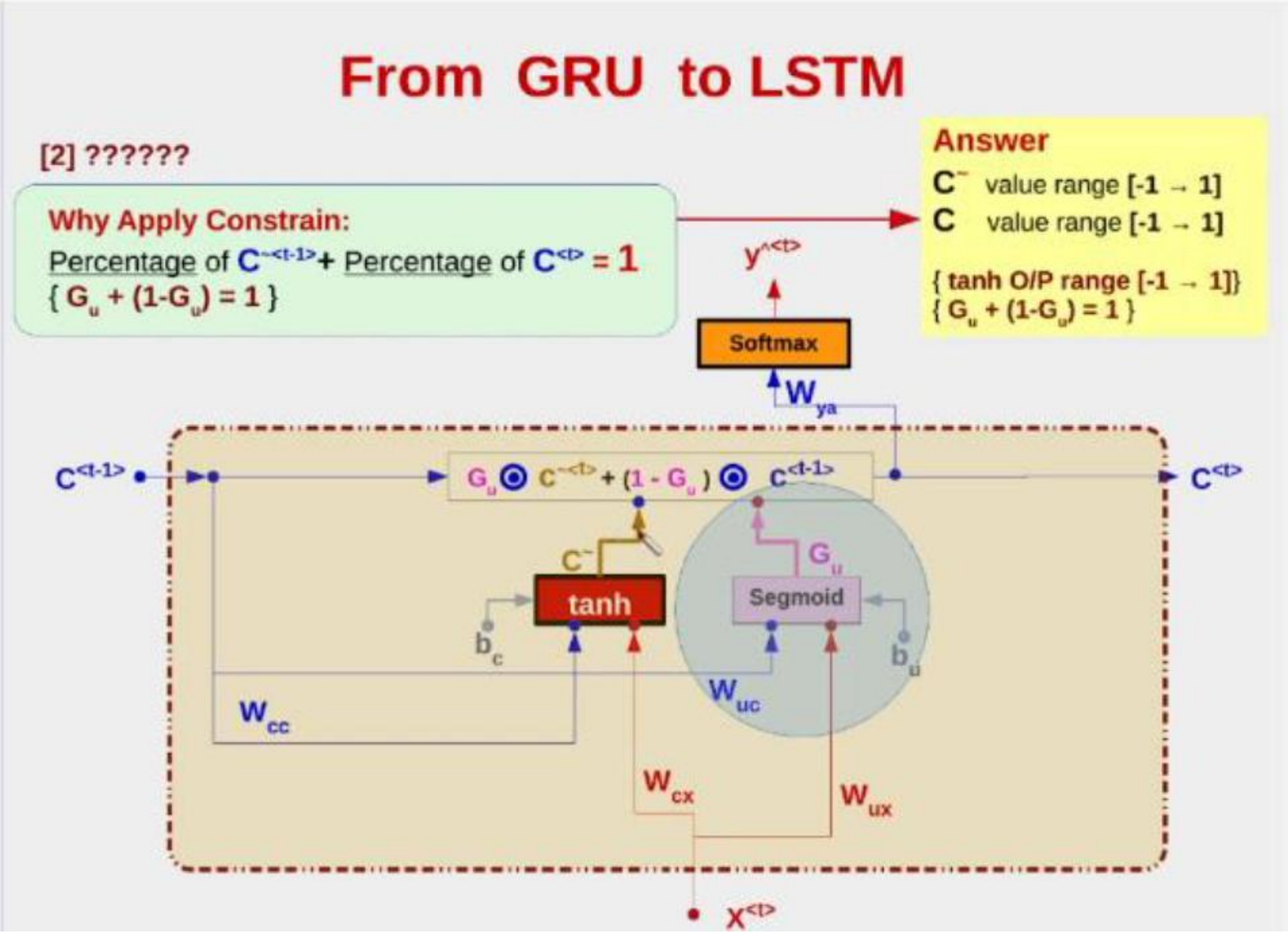
W_{uc} can Do the same functionality
It can Control the amount of relevance of $C^{<t-1>}$



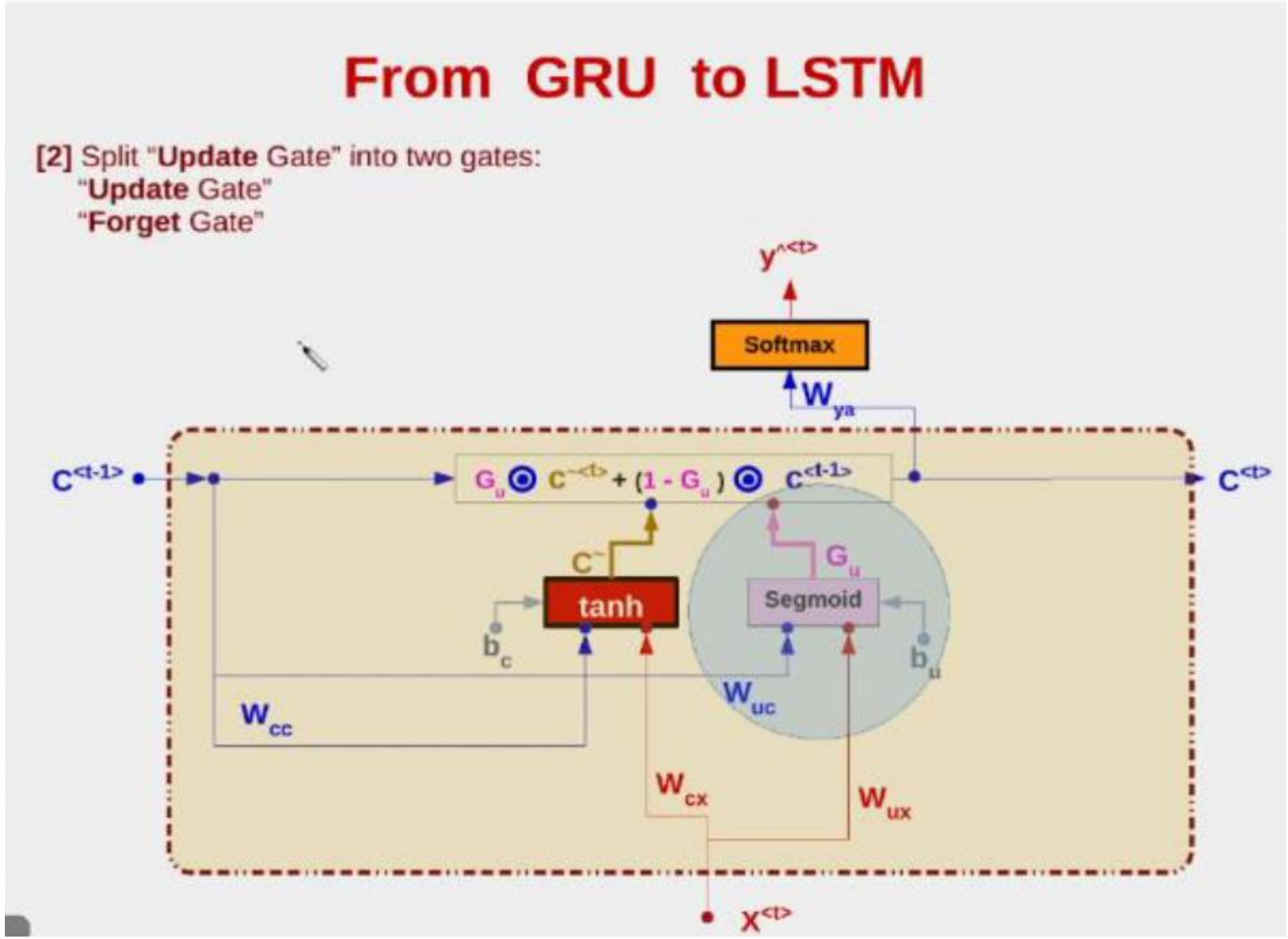
Recurrent Neural Network



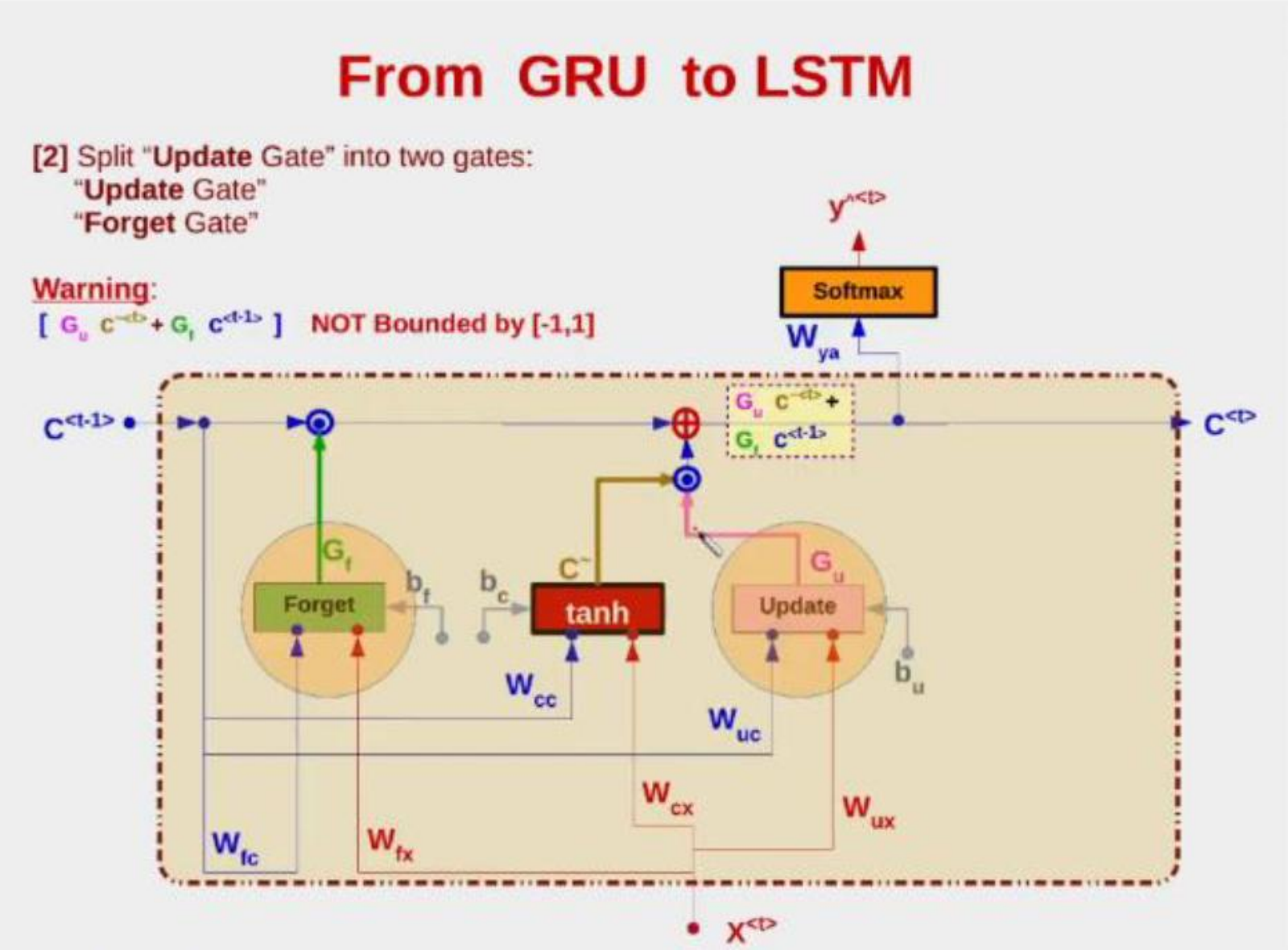
Recurrent Neural Network



Recurrent Neural Network



Recurrent Neural Network



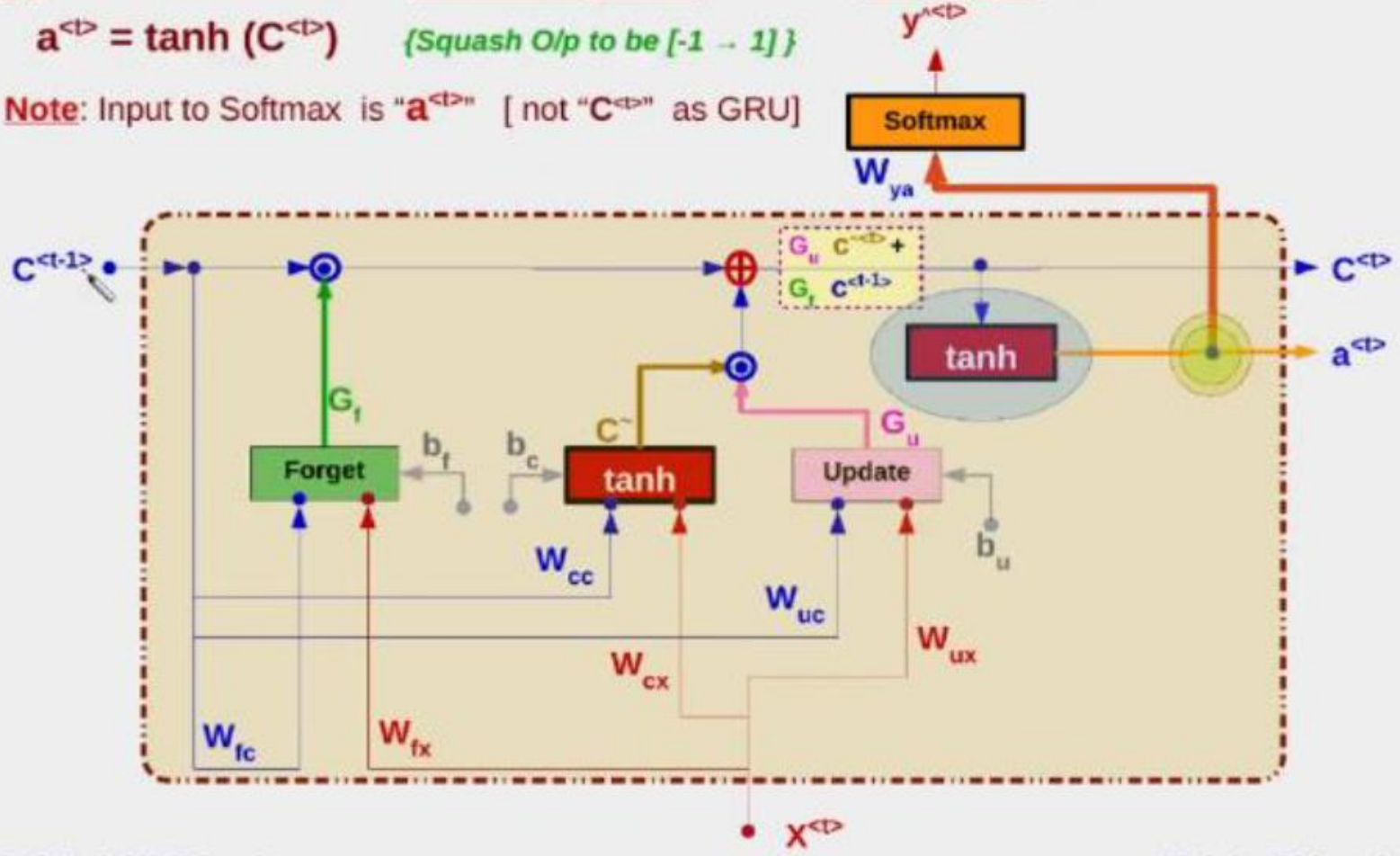
Recurrent Neural Network

From GRU to LSTM

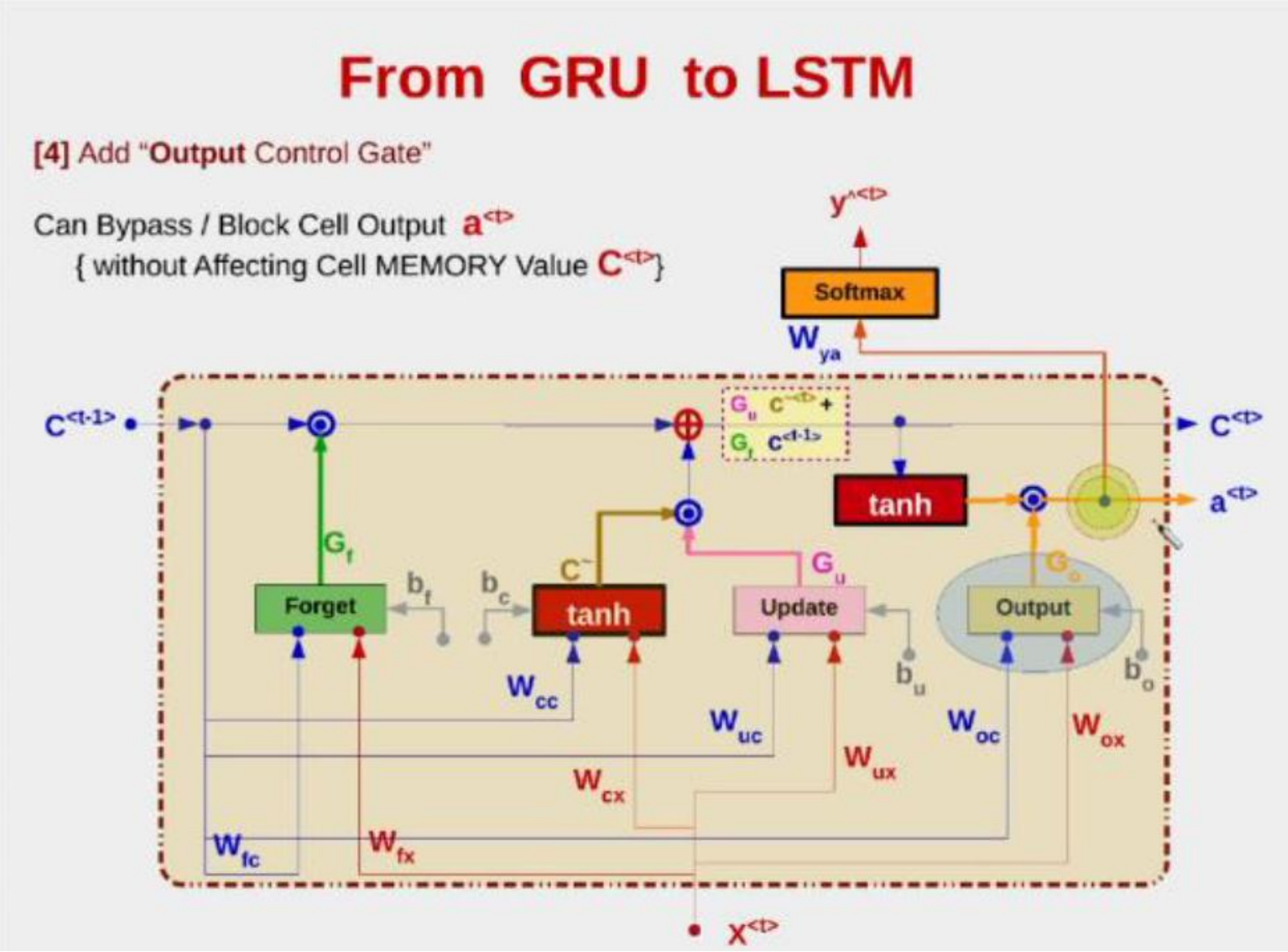
[3] Differentiate between Cell Memory value $C^{<t>}$ and Cell Output $a^{<t>}$

$a^{<t>} = \tanh(C^{<t>})$ *{Squash O/p to be [-1 → 1] }*

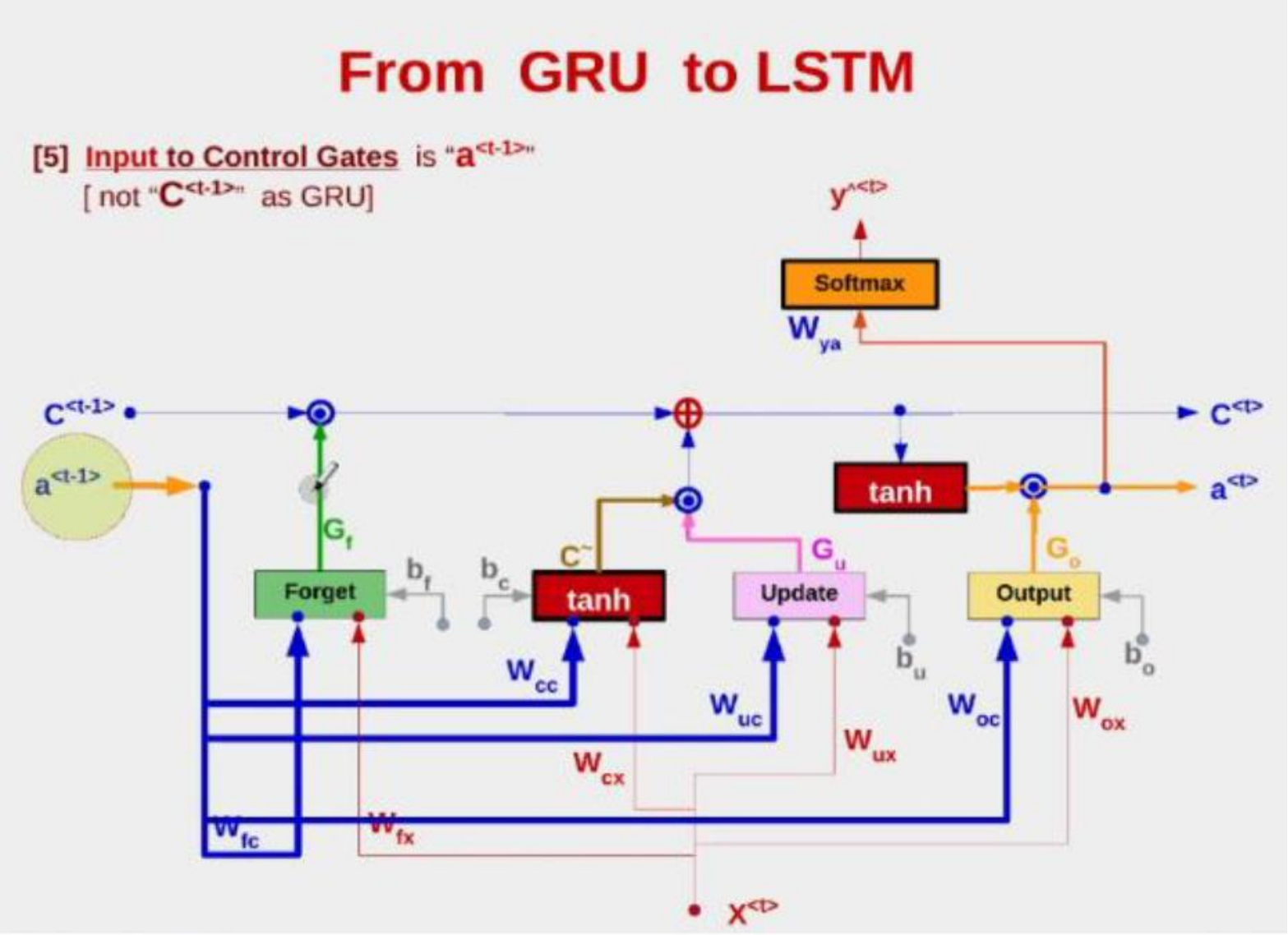
Note: Input to Softmax is "$a^{<t>}$" [not "$C^{<t>}$" as GRU]



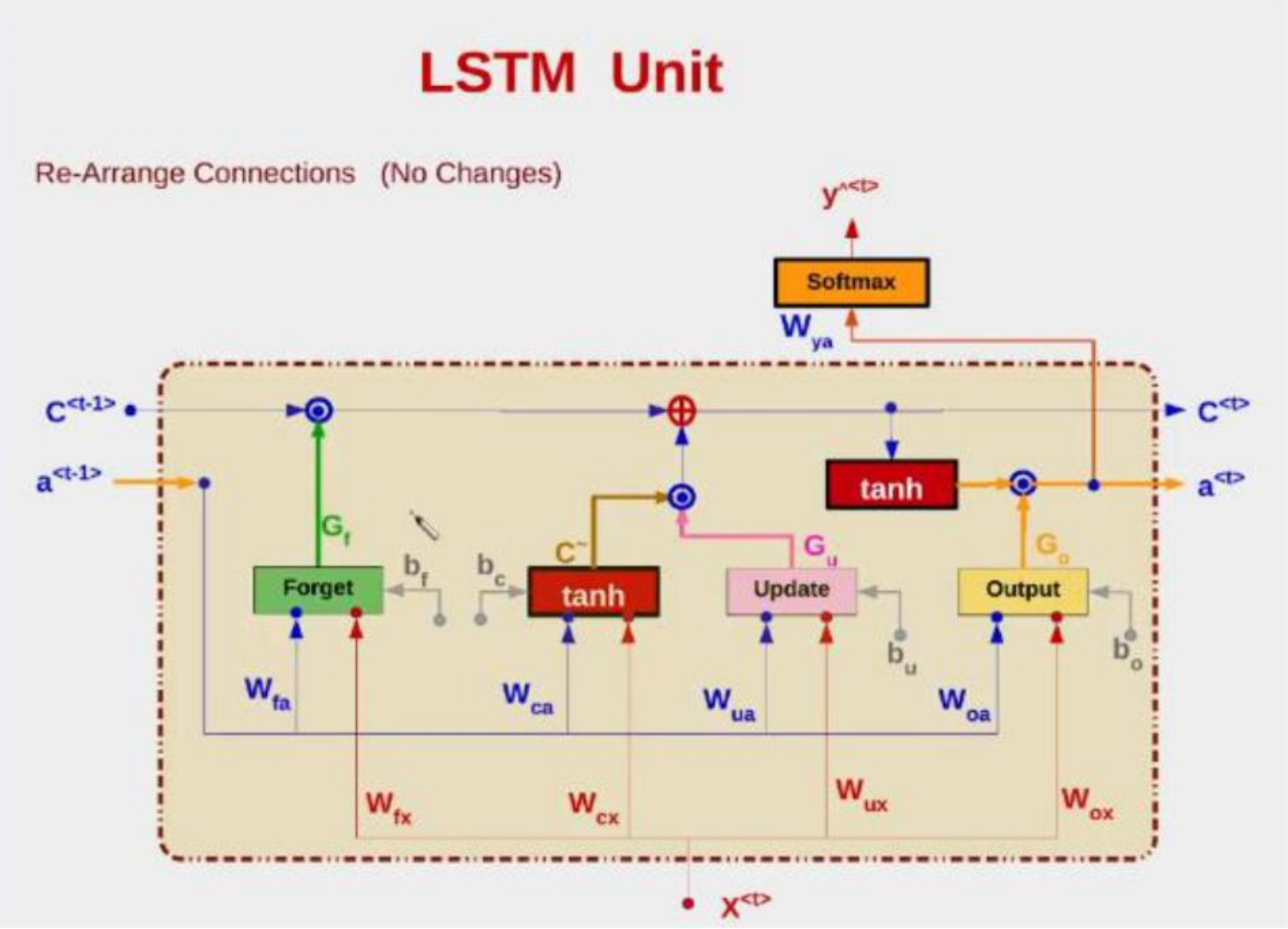
Recurrent Neural Network



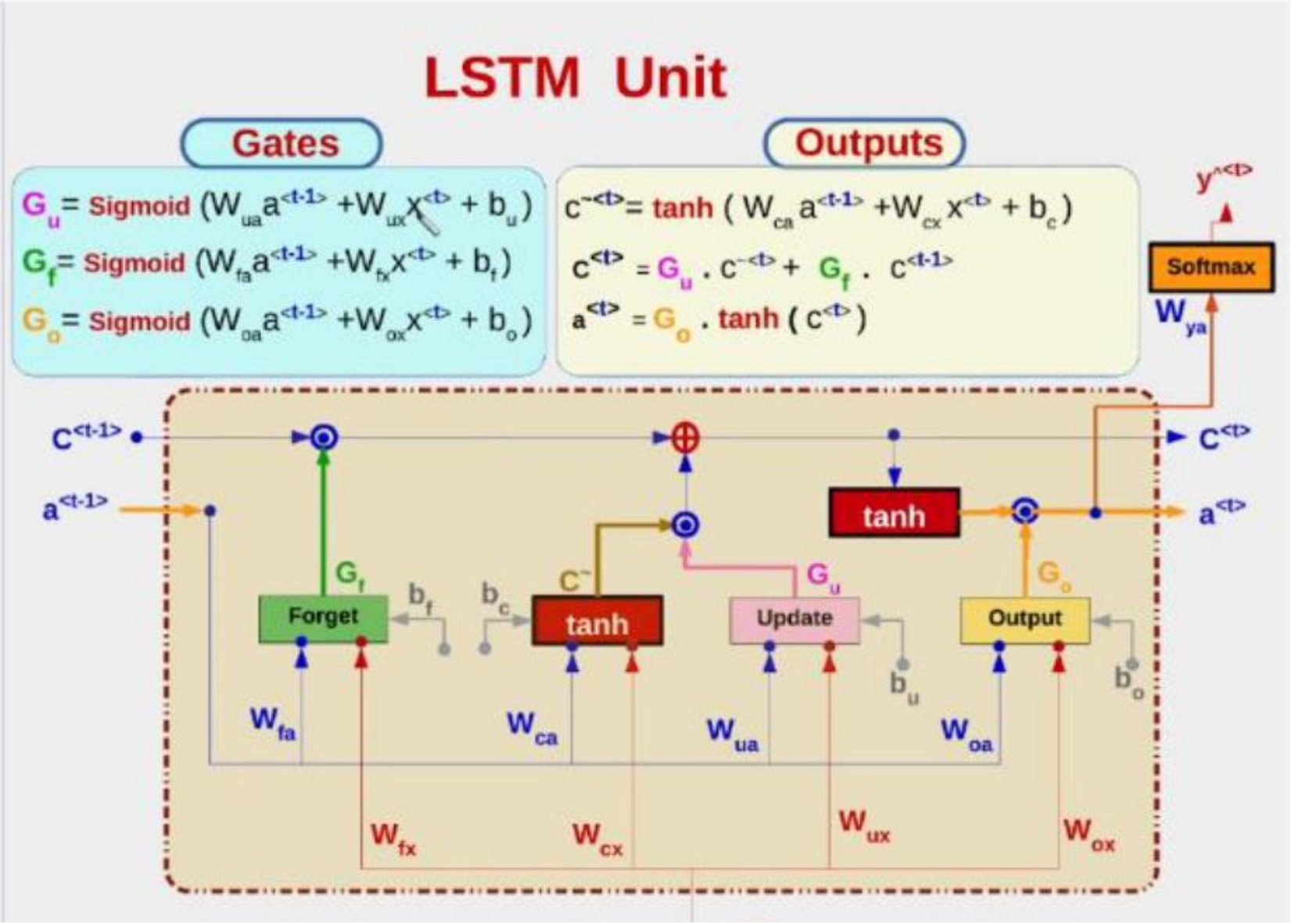
Recurrent Neural Network



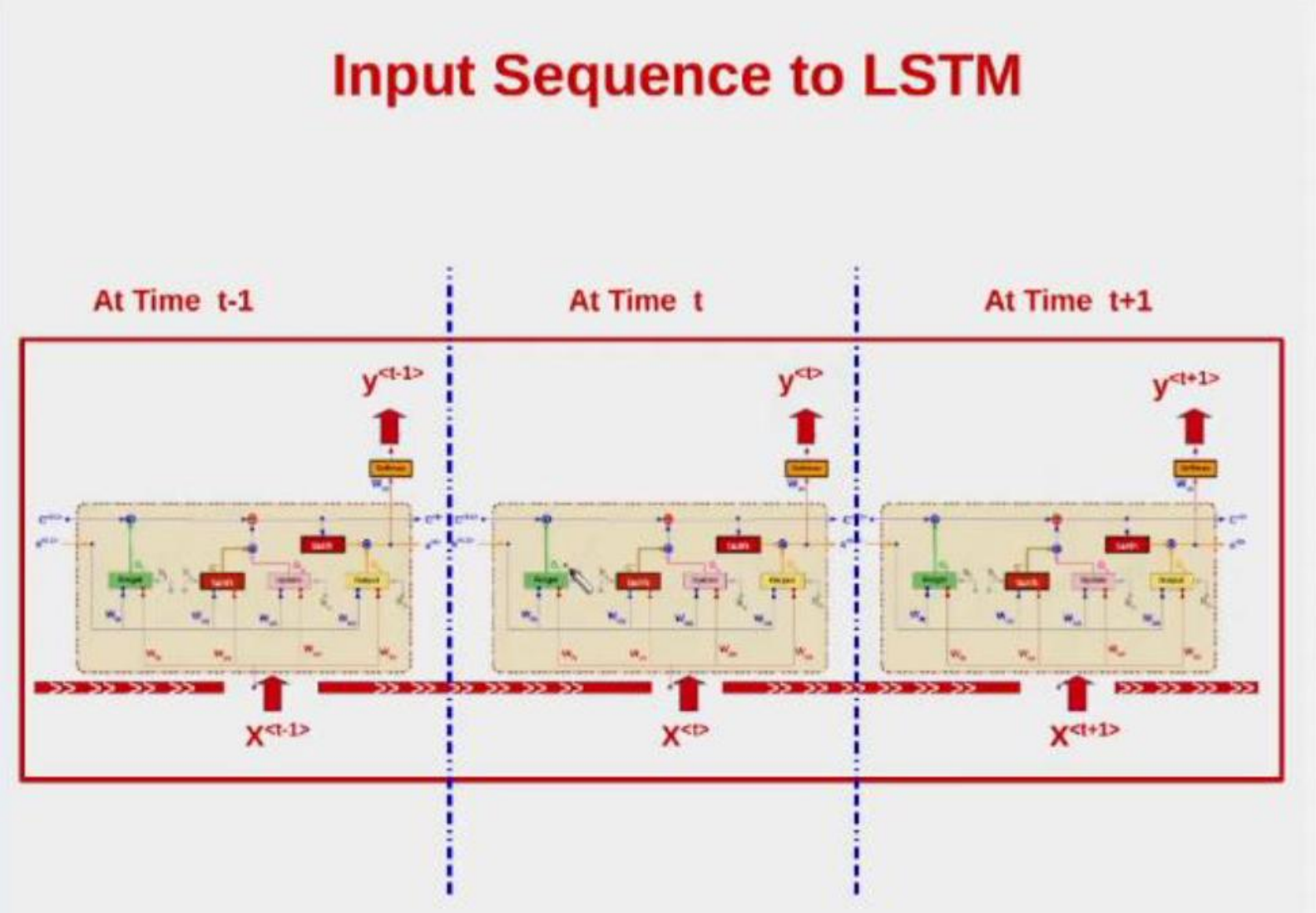
Recurrent Neural Network



Recurrent Neural Network



Recurrent Neural Network



Thank you